Mathematic Foundations for Computer Science

Probability and Optimization

Chapter 4: Optimization

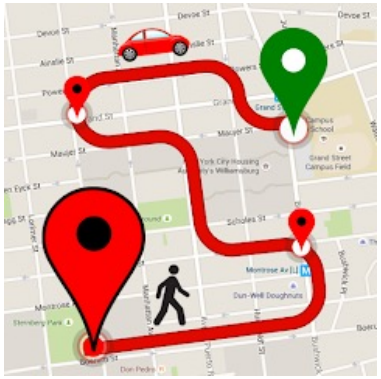
Spring 2025

Instructor: Xiaodong Gu

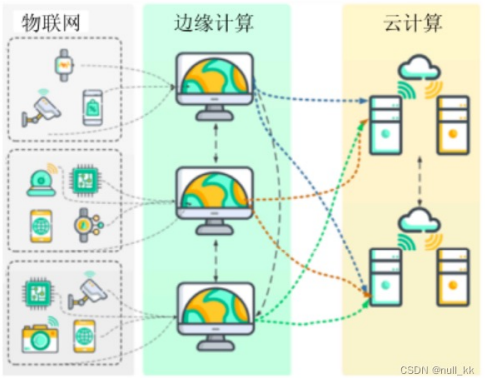


Optimization in Computer Science

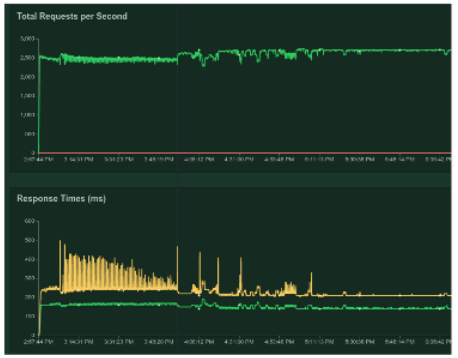
多点路径规划



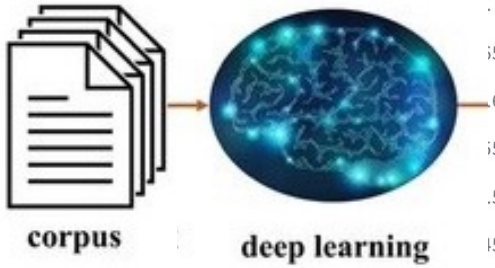
“云-边-端”资源调度优化



操作系统调度

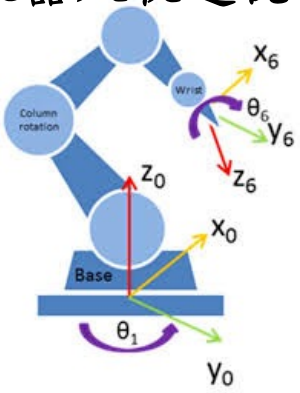


人工智能训练





机器人轨迹优化



Optimization in Computer Science

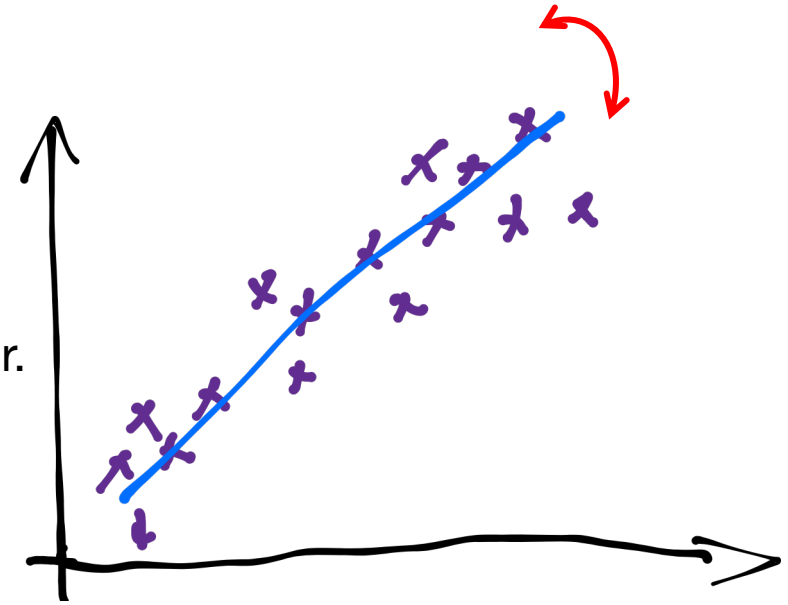
Example: Least squares regression

Given a set of data points $(x_1, r_1), \ldots, (x_m, r_m)$, where $x_i \in R^n$ is the i-th input and $r_i \in R$ is the i-th output, find a linear function $f: R^n \to R$ that best explain the input-output relationship.

Find $f(x) = w^T x$ for some $w \in \mathbb{R}^n$

Objective: minimize mean-square error.

$$\text{minimize}_w \ \|Wx - r\|_2$$

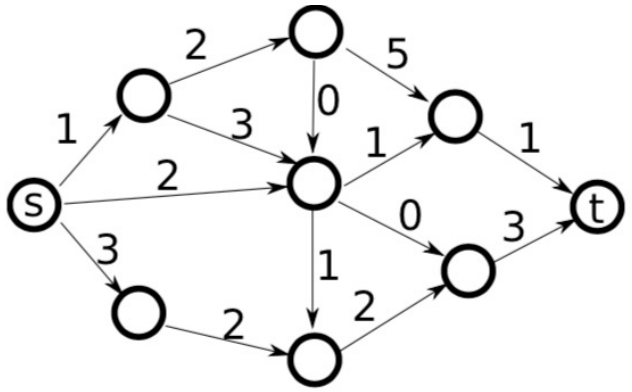


Optimization in Computer Science

Example: shortest path (最短路径)

Given a directed network G = (V, E), with a length of c_e for each edge e, find the shortest path from s to t.

$$x_e = \begin{cases} 1 & \text{edge e is selected} \\ 0 & \text{edge e is not selected} \end{cases}$$



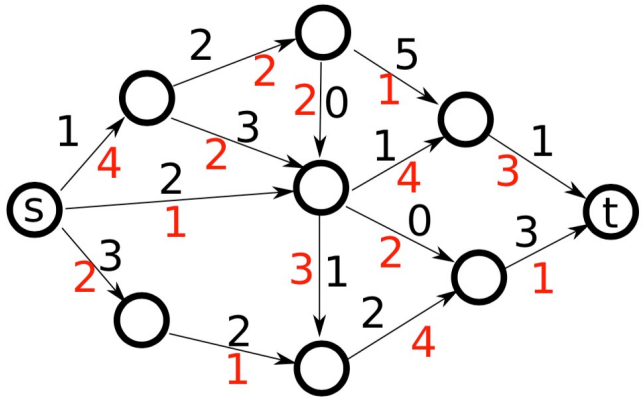
$$\begin{aligned} \text{minimize} \quad & \textstyle\sum_{e \in E} c_e x_e \\ \text{subject to} \quad & \textstyle\sum_{e \leftarrow v} x_e = \sum_{e \rightarrow v} x_e, && \text{for } v \in V \setminus \{s, t\}. \\ & \textstyle\sum_{e \leftarrow s} x_e = 1 \\ & x_e \leq 1, && \text{for } e \in E. \\ & x_e \geq 0, && \text{for } e \in E. \end{aligned}$$

Optimization in Computer Science

Example: Minimum Cost Flow (网络流)

Given a directed network G = (V, E) with cost $c_e \in R^+$ per unit of traffic on edge e, and capacity d_e, find the minimum cost routing of *r* divisible units of traffic from s to t.

$$x_e = \begin{cases} 1 & \text{edge e is selected} \\ 0 & \text{edge e is not selected} \end{cases}$$

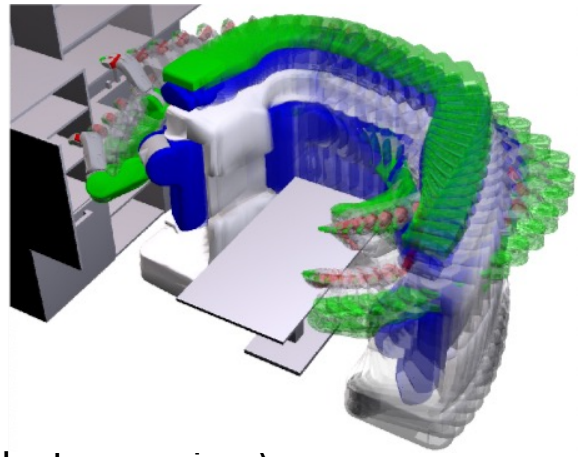


$$\begin{aligned} \text{minimize} \quad & \textstyle\sum_{e \in E} c_e x_e \\ \text{subject to} \quad & \textstyle\sum_{e \leftarrow v} x_e = \sum_{e \rightarrow v} x_e, && \text{for } v \in V \setminus \{s, t\}. \\ & \textstyle\sum_{e \leftarrow s} x_e = r \\ & x_e \leq d_e, && \text{for } e \in E. \\ & x_e \geq 0, && \text{for } e \in E. \end{aligned}$$

Optimization in Computer Science

Example: robot trajectory planning

Figure from (Schulman et al., 2013)



Given robot state x_t and control inputs u_t

$$\underset{x_{1:T}, u_{1:T-1}}{\text{minimize}} \quad \sum_{i=1}^{T-1} \|x_t - x_{t+1}\|_2^2 + \|u_t\|_2^2$$

$$\text{subject to} \quad x_{t+1} = f_{\text{dynamics}}(x_t, u_t), \quad \text{(robot dynamics)}$$

$$f_{\text{collision}}(x_t) \geq 0.1 \quad \text{(avoid collisions)}$$

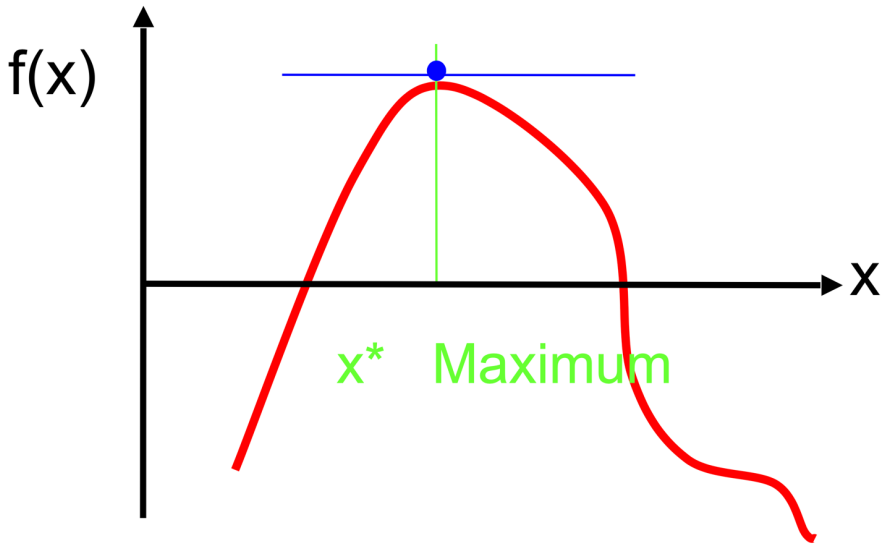
$$x_1 = x_{\text{init}}, \quad x_T = x_{\text{goal}}$$

Schulman, John, et al. "Finding locally optimal, collision-free trajectories with sequential convex optimization." *Robotics: science and systems*. Vol. 9. No. 1. 2013.

What is Optimization?

- Optimization = search for the best solution !
- In mathematical terms:

minimization or **maximization** of an **objective function** $f(x)$ depending on variables x subject to constraints



Mathematical Formulation



Mathematical Optimization

The task of selecting the "best" configuration of a set of variables from a "feasible" set of configurations.

$$\begin{array}{ll} \text{minimize (or maximize)} & f(x) \\ \text{subject to} & x \in \chi \end{array}$$

Overview of Chapter 4

Optimization Problems:

LP, QP, QCQP, GP

Optimization Algorithms:

Gradient descend, Newton's method, interior point method

Optimization Applications:

Resource Allocation, Image Segmentation, Portfolio Optimization



Linear Algebra Warmup

Vector, Matrix, Positive (semi-) definite

Vector

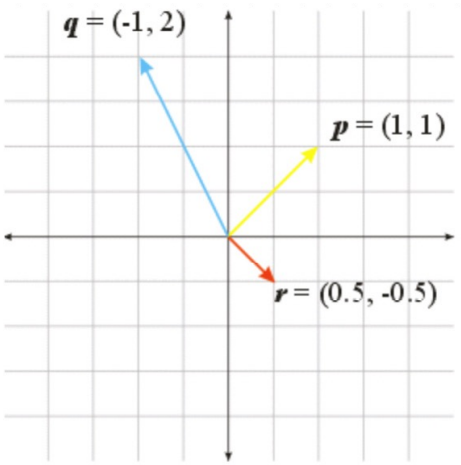
An n-tuple of values where n referred to as the dimension.

- Written as:

$$x = \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix} \qquad x^\top = \begin{pmatrix} x_1 & \cdots & x_n \end{pmatrix}$$

$^\top$ means "transpose"

- Can think of a vector as
  - a point in space, or
  - a directed line segment with a magnitude and direction.



Vector Arithmetic

- Addition of two vectors

$$\boldsymbol{z} = \boldsymbol{x} + \boldsymbol{y} = (x_1 + y_1, \ldots, x_n + y_n)^T$$

- Scalar multiplication of a vector

$$\boldsymbol{y} = a\boldsymbol{x} = (ax_1, \ldots, ax_n)^T$$

Inner Product

- The inner product (dot product) of two vectors $x, y \in R^n$:

$$\boldsymbol{x} \cdot \boldsymbol{y} = \boldsymbol{x}^T\boldsymbol{y} = \boldsymbol{y}^T\boldsymbol{x} = \Sigma_i\, x_i y_i$$

Example: $x = (4, 6, 1)^T$, $y = (1, 3, 1)^T$, $x^Ty = 4\times1+6\times3+1\times1 = 23$

- If $\boldsymbol{x}^T\boldsymbol{y} = 0$, then x and y are orthogonal.

Vector Norms (范数)

For a vector $\boldsymbol{x} \in \mathbb{R}^n$ with elements $\boldsymbol{x}=(x_1, x_2, \ldots, x_n)$:

- The l_2 norm, or Euclidean norm:

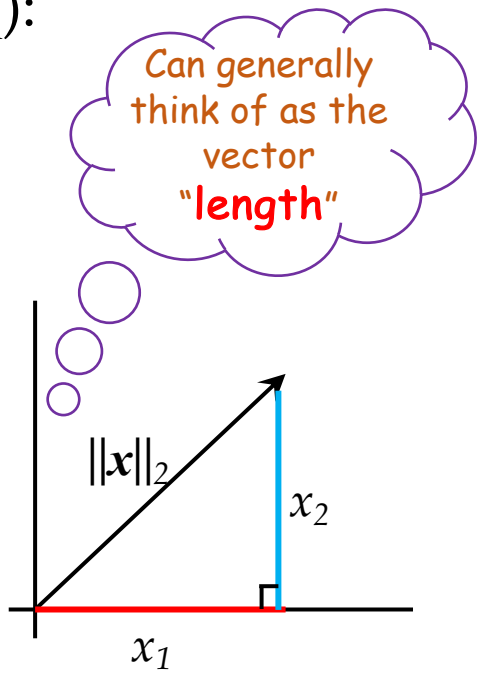
$$\|x\|_2 = \|x\| = \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2}$$

- The l_1 norm:

$$\|x\|_1 = |x_1| + |x_2| + \cdots + |x_n|$$

- The l_p-norm:

$$\|x\|_p = \sqrt[p]{|x|_1^p + |x|_2^p + \cdots + |x|_n^p}$$



https://zhuanlan.zhihu.com/p/26884695

Matrix

- A matrix $\mathbf{A} \in \boldsymbol{R}^{m \times n}$ is a two-dimensional array of values with m rows and n columns.

$$A = \begin{bmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{m1} & \cdots & a_{mn} \end{bmatrix}$$

  - a_{ij} (or A_{ij}) denotes the element in the i-th row and j-th column;
  - a_{i} refers to the i-th column and $a^{\mathrm{T}}{}_{\mathrm{j}}$ refers to the j-th row.

$$A = \begin{bmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{m1} & \cdots & a_{mn} \end{bmatrix} = \begin{bmatrix} | & & | \\ a_1 & \cdots & a_n \\ | & & | \end{bmatrix} = \begin{bmatrix} - & a_1^T & - \\ & \vdots & \\ - & a_m^T & - \end{bmatrix}$$

Matrix Addition/Subtraction

- The addition (subtraction) of two matrices is defined as the addition (subtraction) of corresponding elements.

for $A, B \in \boldsymbol{R}^{m \times n}$, $C \in R^{m \times n} = A + B \Longleftrightarrow c_{ij} = a_{ij} + b_{ij}$

Transpose

- **Vector Transpose**: switches the row and column indices of the vector $\boldsymbol{x}$, often denoted by $\boldsymbol{x}^{\mathrm{T}}$

- **Matrix Transpose**: switches the row and column indices of the matrix $\mathbf{A}$, often denoted by $\mathbf{A}^{\mathrm{T}}$

$$\mathbf{A} = \begin{pmatrix} 2 & 7 & -1 & 0 & 3 \\ 4 & 6 & -3 & 1 & 8 \end{pmatrix} \qquad \mathbf{A}^{\top} = \begin{pmatrix} 2 & 4 \\ 7 & 6 \\ -1 & -3 \\ 0 & 1 \\ 3 & 8 \end{pmatrix}$$

Multiply a Vector by a Matrix

- Multiplying a matrix $\boldsymbol{A} = [a_{ij}]_{m\times n}$ with a vector $\boldsymbol{x} = (x_1,\ldots,x_n)$ can be written as a weighted sum of $\boldsymbol{A}$'s column vectors

$$\mathbf{Ax} = \mathbf{y}$$

$$\begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix}$$

column vector

$$y_i = \sum_{j=1}^{n} a_{ij} x_i$$

Matrix Multiply Vector (Left)

$$\boldsymbol{x}^{\mathrm{T}}\mathbf{A} = \mathbf{y}$$

$$[x_1, x_2, \ldots, x_n] \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix} = [y_1, y_2, \ldots, y_n]$$

row vector

$$y_j = \sum_{i=1}^{m} a_{ij} x_j$$

Matrix Multiply Vector (Left & Right)

$$\mathbf{x}^{\mathrm{T}}\mathbf{A}\mathbf{x} = z$$

$$[x_1, x_2, \ldots, x_n] \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

$$= [y_1, y_2, \ldots, y_n] \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \sum_{i=1}^{n} y_i x_i = z$$

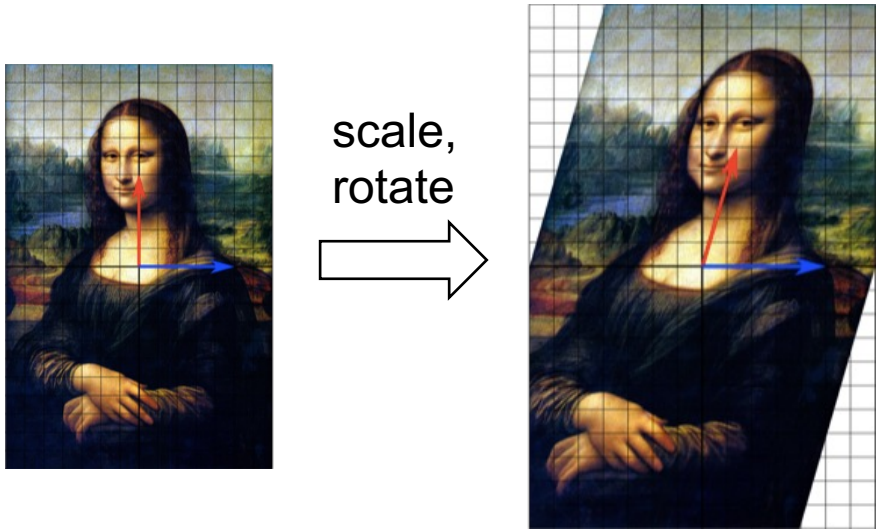
salar

Matrix Multiplication

If $\mathbf{C}_{m\times n} = \mathbf{A}_{m\times p}\mathbf{B}_{p\times n}$, then $[c_{ij}] = \sum_{k=1}^{p} a_{ik}b_{kj}$

- in general, non-commutative: $\mathbf{AB} \neq \mathbf{BA}$
- associative: $(\mathbf{AB})\mathbf{C} = \mathbf{A}(\mathbf{BC})$
- distributive: $(\mathbf{A}+\mathbf{B})\mathbf{C} = \mathbf{AC}+\mathbf{BC}$

Matrix multiplication scales/rotates a geometric plane

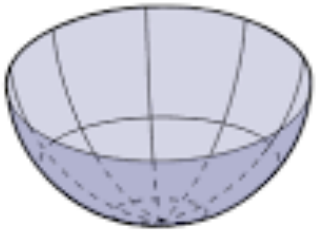


Positive (semi-)definite Matrices

- A symmetric matrix A is positive semi-definite (PSD) if for all $x \in \mathrm{R}^n$, $x^{\mathrm{T}}\mathrm{A}x \geq 0$

  Example:

$$z^T I z = \begin{bmatrix} x & y \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = x^2 + y^2 \geq 0$$



  - The identity matrix $I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ is positive–definite

- A symmetric matrix A is positive definite (PD) if for all $x \in \mathrm{R}^n$, $x^{\mathrm{T}}\mathrm{A}x > 0$

Notation: $\mathbf{A} \succeq 0$ if A is PSD, $\mathbf{A} \succ 0$ if A is PD

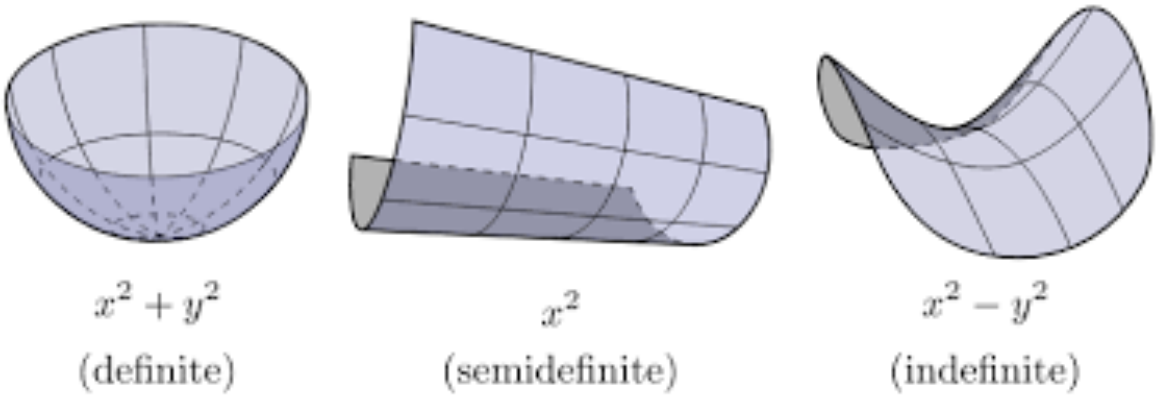
Positive (semi-)definite Matrices

Examples:

$$z^T Az = \begin{bmatrix} x & y \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = x^2 + y^2$$

$$z^T Az = \begin{bmatrix} x & y \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = x^2$$

$$z^T Az = \begin{bmatrix} x & y \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = x^2 - y^2$$



$x^2 + y^2$ (definite)

x^2 (semidefinite)

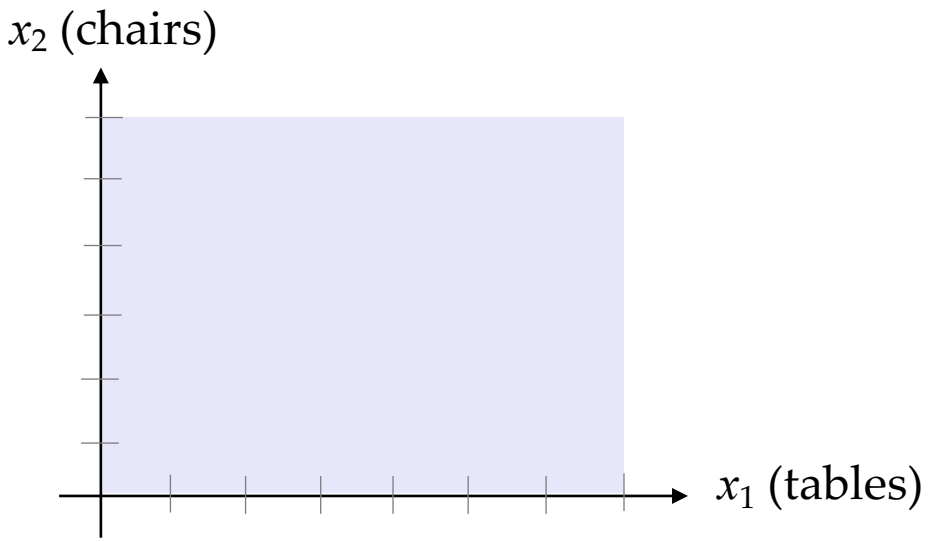
$x^2 - y^2$ (indefinite)



Linear Programming

Example: Optimal Manufacturing

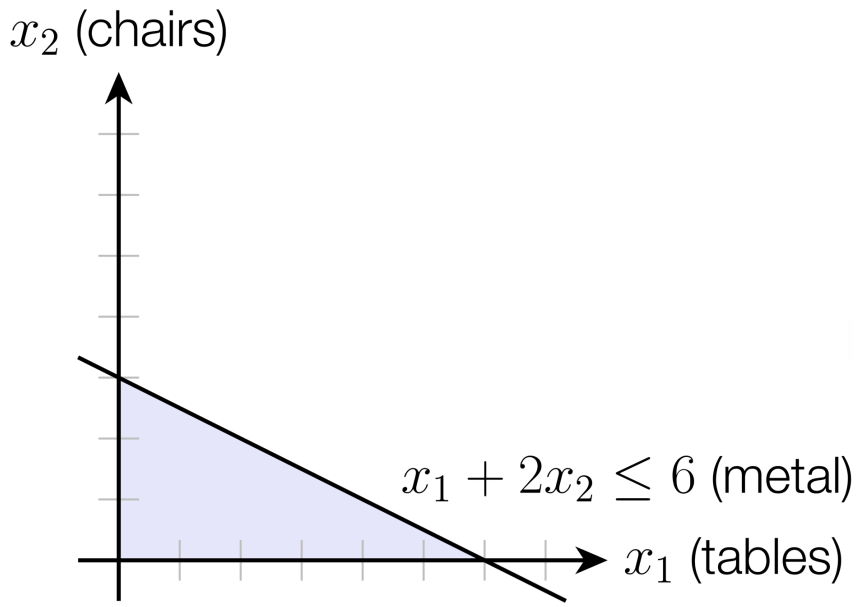
A large factory makes tables and chairs. Each table returns a profit of $200 and each chair a profit of $100. Each table takes 1 unit of metal and 3 units of wood and each chair takes 2 units of metal and 1 unit of wood. The factory has 6K units of metal and 9K units of wood. How many tables and chairs should the factory make to maximize profit?



Example: Optimal Manufacturing

A large factory makes tables and chairs. Each table returns a profit of $200 and each chair a profit of $100. Each table takes 1 unit of metal and 3 units of wood and each chair takes 2 units of metal and 1 unit of wood. The factory has 6K units of metal and 9K units of wood. How many tables and chairs should the factory make to maximize profit?

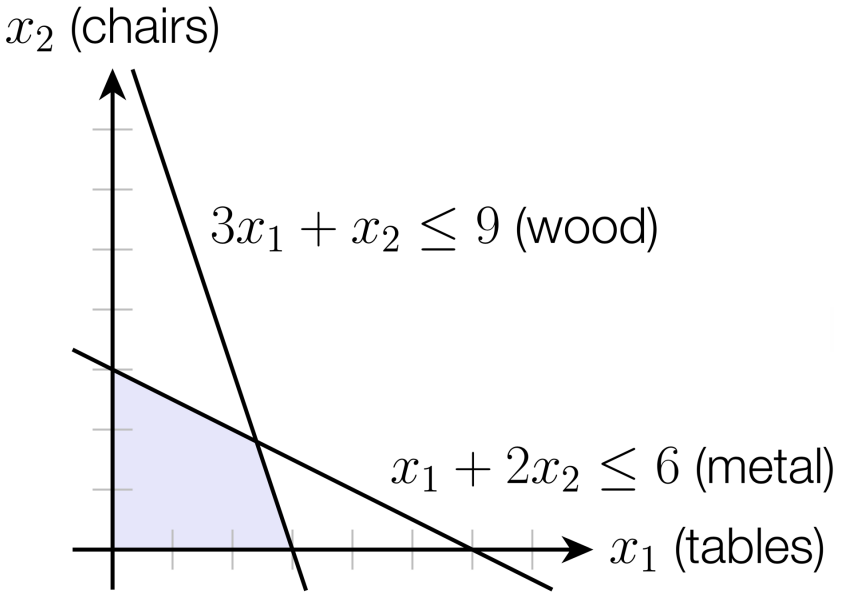
	table x_1	chair x_2	amt.
metal	1	2	6



Example: Optimal Manufacturing

A large factory makes tables and chairs. Each table returns a profit of $200 and each chair a profit of $100. Each table takes 1 unit of metal and 3 units of wood and each chair takes 2 units of metal and 1 unit of wood. The factory has 6K units of metal and 9K units of wood. How many tables and chairs should the factory make to maximize profit?

	table x_1	chair x_2	amt.
metal	1	2	6
wood	3	1	9



Example: Optimal Manufacturing

A large factory makes tables and chairs. Each table returns a profit of $200 and each chair a profit of $100. Each table takes 1 unit of metal and 3 units of wood and each chair takes 2 units of metal and 1 unit of wood. The factory has 6K units of metal and 9K units of wood. How many tables and chairs should the factory make to maximize profit?

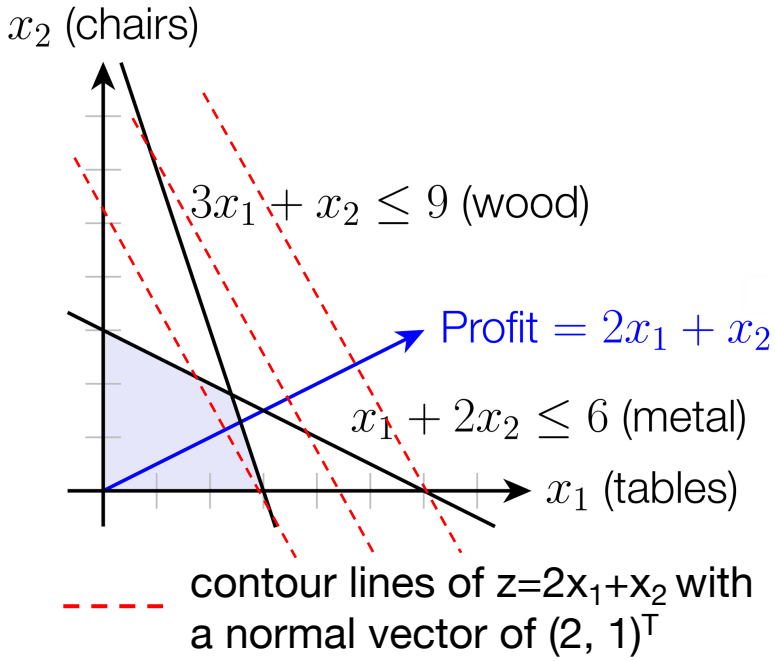


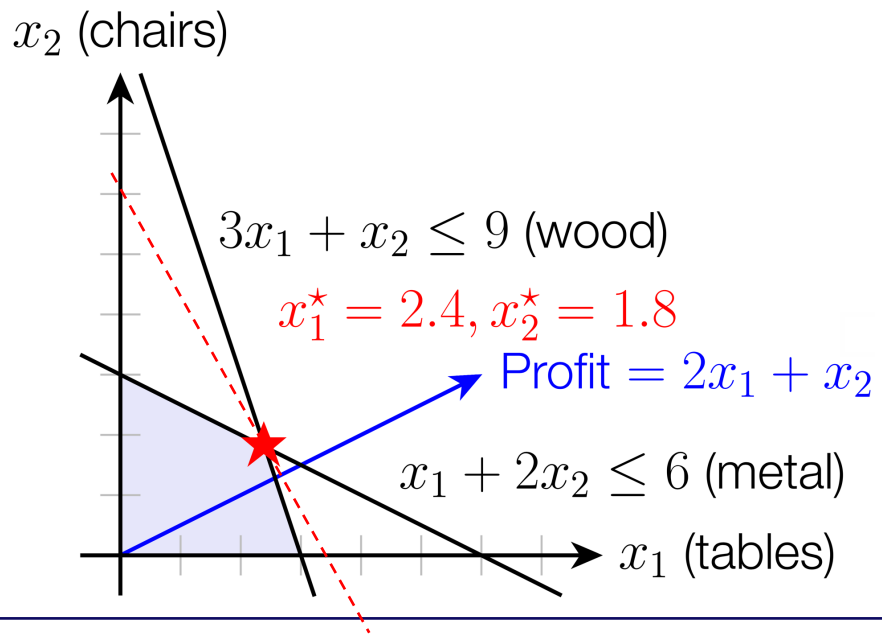
	table x_1	chair x_2	amt.
metal	1	2	6
wood	3	1	9
	$200	$100	

Which contour line stands for the optimal profit?



Example: Optimal Manufacturing

A large factory makes tables and chairs. Each table returns a profit of $200 and each chair a profit of $100. Each table takes 1 unit of metal and 3 units of wood and each chair takes 2 units of metal and 1 unit of wood. The factory has 6K units of metal and 9K units of wood. How many tables and chairs should the factory make to maximize profit?



$$\begin{array}{ll} \underset{x_1, x_2}{\text{maximize}} & 2x_1 + x_2 \\ \text{subject to} & x_1 + 2x_2 \leq 6 \\ & 3x_1 + x_2 \leq 9 \\ & x_1, x_2 \geq 0 \end{array}$$

$$c = \begin{pmatrix} 2 \\ 1 \end{pmatrix} \quad x = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \quad \Downarrow \quad A = \begin{pmatrix} 1 & 2 \\ 3 & 1 \end{pmatrix} \quad b = \begin{pmatrix} 6 \\ 9 \end{pmatrix}$$

$$\begin{array}{ll} \text{maximize} & c^T x \\ \text{subject to} & Ax \preceq b \\ & x \succeq 0 \end{array}$$

Example: Manufacturing

The generic form

- n products, m raw materials
- Every unit of product j uses a_{ij} units of raw material i
- There are b_i units of material i available
- Product j yields profit c_j per unit
- Facility wants to maximize profit subject to available raw materials

x_1	x_2	x_3	x_4	
a_{11}	a_{12}	a_{13}	a_{14}	b_1
a_{21}	a_{22}	a_{23}	a_{24}	b_2
a_{31}	a_{32}	a_{33}	a_{34}	b_3
c_1	c_2	c_3	c_4	

maximize $z = c_1x_1+c_2x_2+\ldots+c_nx_n$

subject to $a_{11}x_1+a_{12}x_2+\ldots+a_{1n}x_n \leq b_1$

$\ldots$

$a_{m1}x_1+a_{m2}x_2+\ldots+a_{mn}x_n \leq b_m$

and $x_i \geq 0$, for $i=1,\ldots,n$

Linear Programming: Canonical Form

$$\begin{aligned} \text{maximize} \quad & c^T x \\ \text{subject to} \quad & a_i^T x \leq b_i, \quad \text{for } i = 1, \ldots, m \\ & x_j \geq 0, \quad \text{for } j = 1, \ldots, n \end{aligned}$$

- Every LP can be transformed to the canonical form (规范形式):
  - Minimizing $c^T x$ is equivalent to maximizing $-c^T x$
  - ≥ constraints can be flipped by multiplying by -1
  - Each equality constraint can be replaced by two inequalities ($a_i^T x \leq b_i$ and $-a_i^T x \leq -b_i$)
  - Unconstrained variables x_j can be replaced by $x_j^+ - x_j^-$, where both x_j^+ and x_j^- are constrained to be nonnegative.

Example:

minimize $3x_1+x_2$
subject to $x_1 > x_2+5$
$x_1+3x_2=10$

⇒

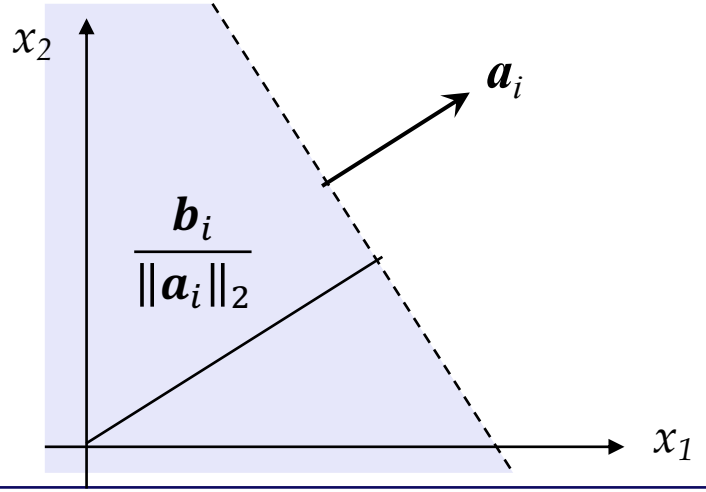
maximize $-3x_1-x_2$
subject to $-x_1+x_2 < -5$
$x_1+3x_2 \leq 10$
$-x_1-3x_2 \leq 10$

Geometric View

- Consider the inequality constraints :

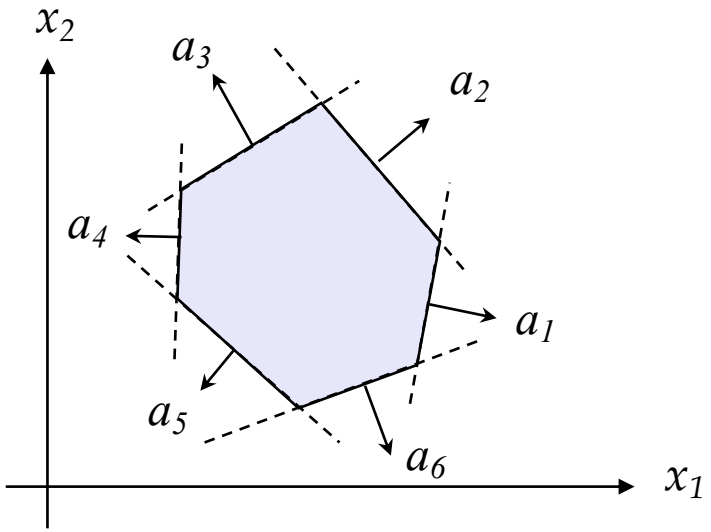
$$\begin{aligned} \underset{x}{\text{maximize}} \quad & c^T x \\ \text{subject to} \quad & a_i^T x \le b_i, \ i = 1, \ldots, m \end{aligned}$$

- Each linear inequality constraint $a_i^{\mathrm{T}} x \le b_i$ defines a half-space, which is orthogonal to the coefficient vector a.



Geometric View

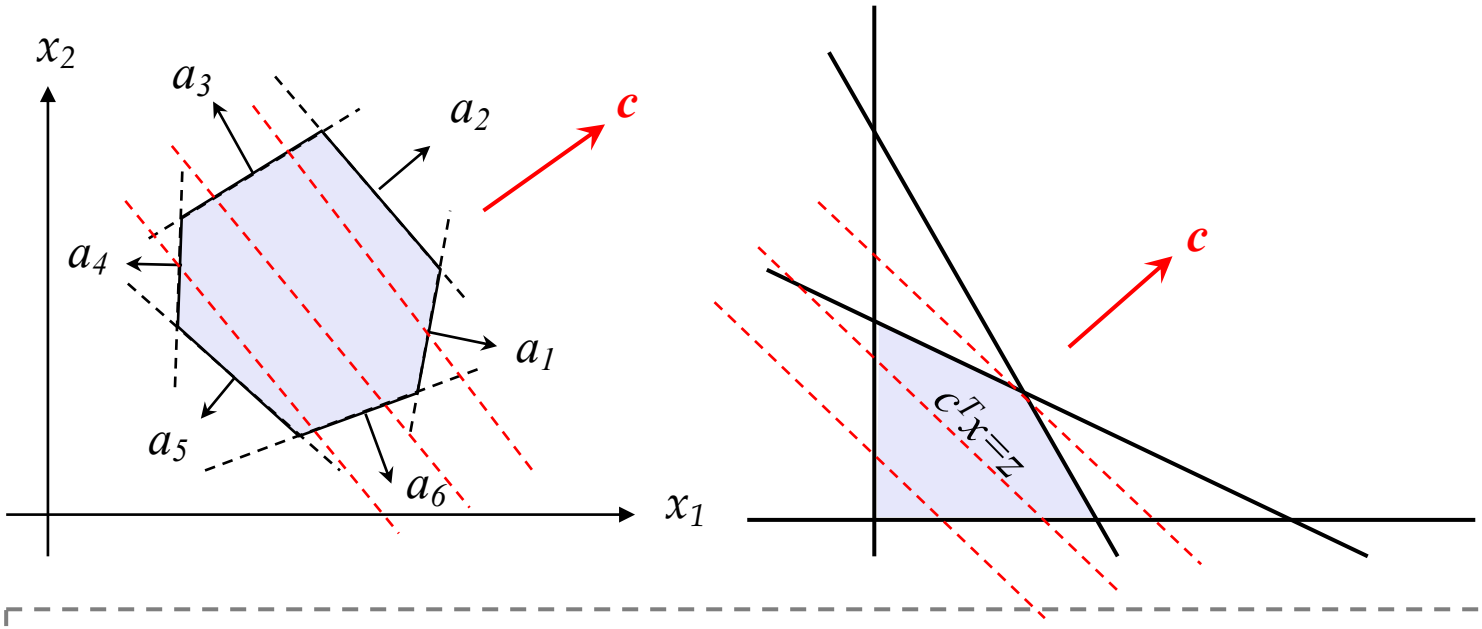
- Multiple halfspace constraints, $a_i^T x \le b_i$, $i = 1, \ldots, m$ (or equivalently $Ax \le b$), define what is called a polyhedron (多面体).



The feasible region of an LP is a polyhedron.

Geometric View

- So linear programming is equivalent to maximizing a contour toward a direction (c) over a polyhedron.

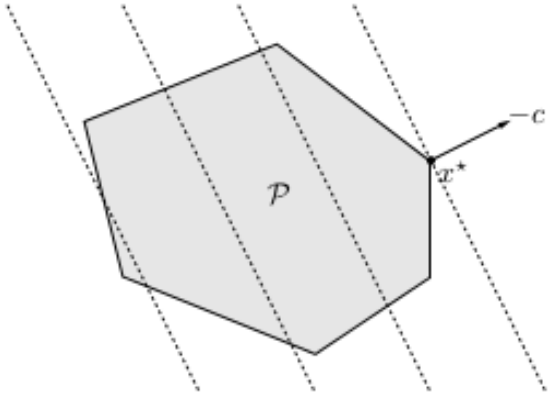


Important note: the optimal solution always occur at a "corner" of the polyhedron (this is exactly the property that the simplex algorithm will exploit)

In Summary

- We have already seen linear programming

$$\begin{aligned} \text{minimize} \quad & \boldsymbol{c}^{\mathrm{T}}\boldsymbol{x} \\ \text{subject to} \quad & \boldsymbol{A}\boldsymbol{x} \leq \boldsymbol{b} \\ & \boldsymbol{x} \geq 0 \end{aligned}$$



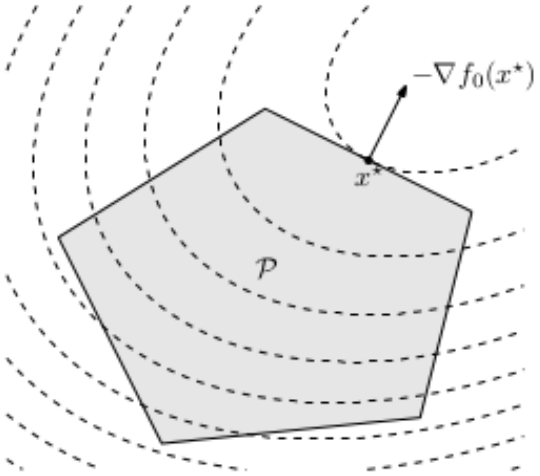


Any other optimization problems?

Quadratic Programming (QP) 二次规划

- Minimize a convex quadratic function (二次函数) over a polyhedron. Require $P \succcurlyeq 0$.

$$\begin{array}{ll} \text{minimize} & x^\intercal P x + c^\intercal x + d \\ \text{subject to} & Ax \leq b \end{array}$$



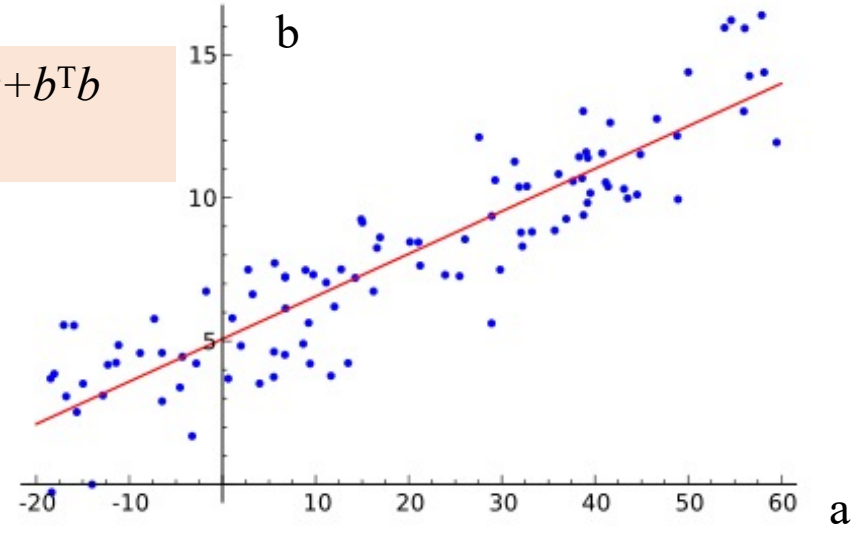
$P \succcurlyeq 0$: positive semi-definite, $P \in S^n_+$
symmetric & all eigenvalues are nonnegative,
$x^T \boldsymbol{P} x \geq 0$ for all x

Case: Least Squares Regression

带约束的最小二乘

Given a set of measurements $(a_1, b_1), \ldots, (a_m, b_m)$, where $a_i \in \mathrm{R}^n$ is the i-th input and $b_i \in \mathrm{R}$ is the i-th output, fit a linear function minimizing mean square error, subject to known bounds on the linear coefficients.

$$\text{minimize } ||\boldsymbol{A}x - b||^2_2 = x^T\boldsymbol{A}^T\boldsymbol{A}x\text{-}2b^T\boldsymbol{A}x+b^Tb$$
$$\text{subject to } l_i \le x_i \le u_i, \text{ for } i = 1,\ldots,n.$$

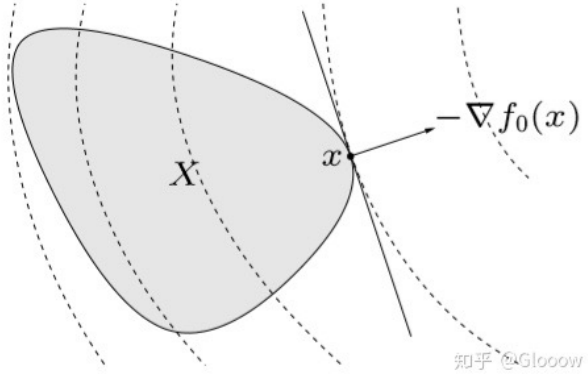


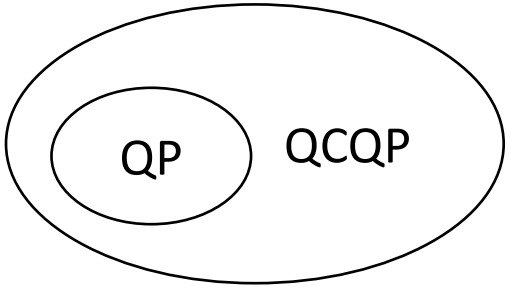
Quadratically Constrained Quadratic Programming (QCQP) 二次约束二次规划

- Minimize a convex quadratic function (二次函数) over a quadratic constraint.

$$
\begin{array}{ll}
\text{minimize} & (1/2)x^T P_0 x + q_0^T x + r_0 \\
\text{subject to} & (1/2)x^T P_i x + q_i^T x + r_i \leq 0, \quad i = 1, \ldots, m \\
& Ax = b
\end{array}
$$

- $P_i \in \mathbf{S}_+^n$; objective and constraints are convex quadratic
- if $P_1, \ldots, P_m \in \mathbf{S}_{++}^n$, feasible set is intersection of m ellipsoids and an affine set



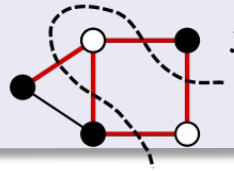


Case: Max Cut (图的最大割问题)

Max Cut Problem (最大割问题)

Given an undirected graph G = (V, E), find a partition of V into (S, V\S) maximizing number of edges with exactly one end in S.

$$\text{maximize} \quad \Sigma_{(i,j)\in E}(1-x_i x_j)/2$$
$$\text{subject to} \quad x_i \in \{-1,1\}, \text{ for } i \in V.$$



$$x_i = \begin{cases} +1 \text{ if } v_i \in S \\ -1 \text{ if } v_i \in V\backslash S \end{cases}$$

Adjacent Matrix Form

$$\text{maximize} \quad 1/4\Sigma_{(i,j)}(1-x_i x_j)G_{ij}$$
$$\text{subject to} \quad x_i^2=1, \text{ for } i \in V.$$

G is the adjacent matrix
$1-x_i x_j=2$ if {i, j} is a cut

Matrix Form

$$\text{minimize} \quad \boldsymbol{x}^T G \boldsymbol{x}$$
$$\text{subject to} \quad x_i^2=1, \quad \text{for } i \in V.$$

Geometric Programming (GP) 几何规划

- Geometric programming is an optimization problem

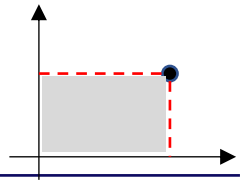
$$\begin{array}{lll} \text{minimize} & f_0(x) & \\ \text{subject to} & f_i(x) \le b_i, & \text{for } i \in \mathcal{C}_1. \\ & h_i(x) = b_i, & \text{for } i \in \mathcal{C}_2. \\ & x \succeq 0 & \end{array}$$

where f_i's are posynomials, h_i's are monomials, and $b_i > 0$ ($w \log 1$).

Definition

- A monomial (单项式) is a function $f: \mathrm{R}^n_+ \to \mathrm{R}_+$ of the form
$$f(x) = c x_1^{a_1} x_2^{a_2} \dots x_n^{a_n}$$
where $c \ge 0$, $a_i \in \mathrm{R}$.
- A posynomial (正多项式) is a sum of monomials.

Interpretation: GP minimizes volume/area, subject to constraints.



Case: Suitcase Design

A manufacturer is designing a suitcase (手提箱)

- variables: h, w, d
- want to minimize surface area: $2(hw + hd + wd)$ (i.e., amount of material used)
- have a target volume: $hwd \geq 5$
- practical/aesthetic (美学) constraints limit aspect ratio:

$$h/w \leq 2,\ h/d \leq 3$$

- constrained by airline to $h + w + d \leq 7$

$$\begin{array}{ll} \text{minimize} & 2hw + 2hd + 2wd \\ \text{subject to} & h^{-1}w^{-1}d^{-1} \leq \frac{1}{5} \\ & hw^{-1} \leq 2 \\ & hd^{-1} \leq 3 \\ & h + w + d \leq 7 \\ & h, w, d \geq 0 \end{array}$$

$$\begin{aligned} \widetilde{h} &= \log h \\ \widetilde{w} &= \log w \\ \widetilde{d} &= \log d \end{aligned}$$

$$\Longrightarrow$$

$$\begin{array}{ll} \text{minimize} & 2e^{\widetilde{h}+\widetilde{w}} + 2e^{\widetilde{h}+\widetilde{d}} + 2e^{\widetilde{w}+\widetilde{d}} \\ \text{subject to} & e^{-\widetilde{h}-\widetilde{w}-\widetilde{d}} \leq \frac{1}{5} \\ & e^{\widetilde{h}-\widetilde{w}} \leq 2 \\ & e^{\widetilde{h}-\widetilde{d}} \leq 3 \\ & e^{\widetilde{h}} + e^{\widetilde{w}} + e^{\widetilde{d}} \leq 7 \end{array}$$

Summary of Optimization Problems

- Find the minimizer of a function subject to constraints:

$$\begin{array}{lll} \underset{x}{\text{minimize}} & f_0(x) & \\ \text{subject to} & f_i(x) \leq 0 & i = 1, \ldots, m \\ & h_i(x) = 0 & i = 1, \ldots, p \end{array}$$

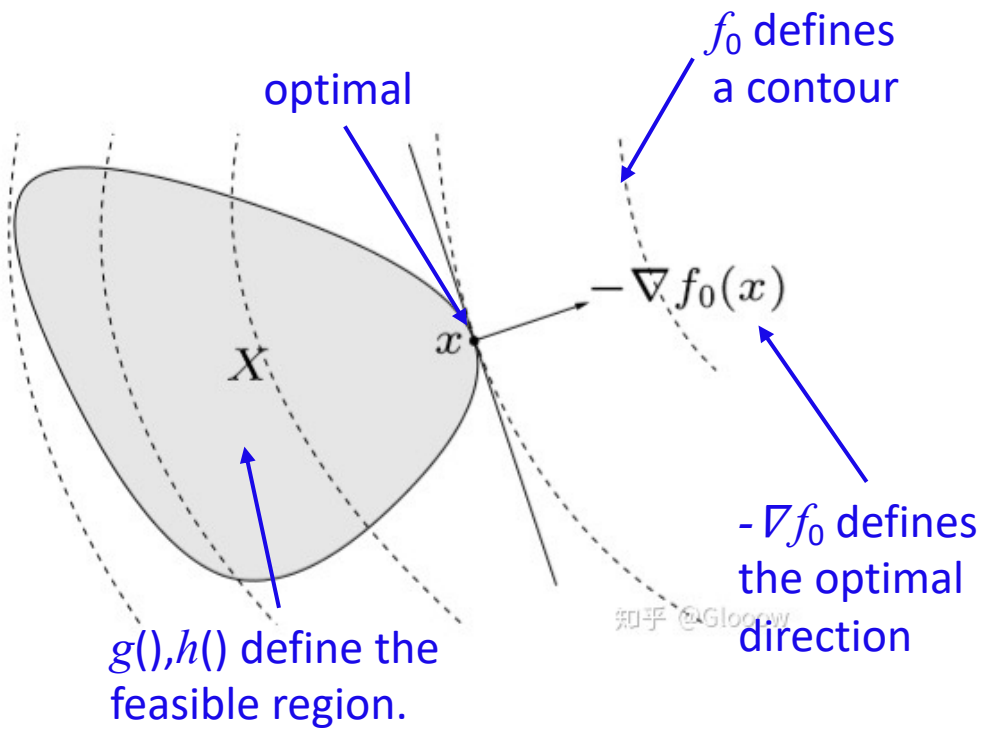
where

$x = (x_1, \ldots, x_n)$ is the optimization variable
$f_0 : \mathbf{R}^n \longrightarrow \mathbf{R}$ is the objective function
$f_i : \mathbf{R}^n \longrightarrow \mathbf{R}, \quad i = 1, \ldots, m$ are inequality constraint functions
$h_i : \mathbf{R}^n \longrightarrow \mathbf{R}, \quad i = 1, \ldots, p$ are equality constraint functions.

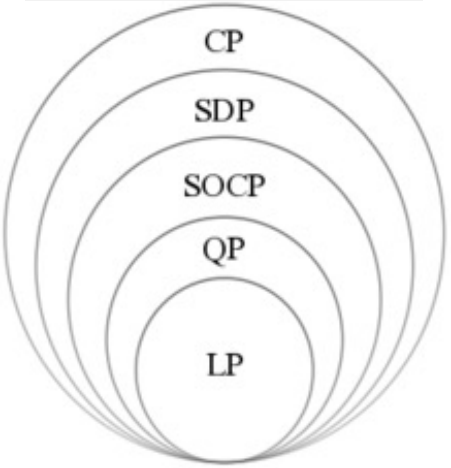
Goal: find an optimal solution $x^\star$ that minimizes f_0 while satisfying all the constraints.

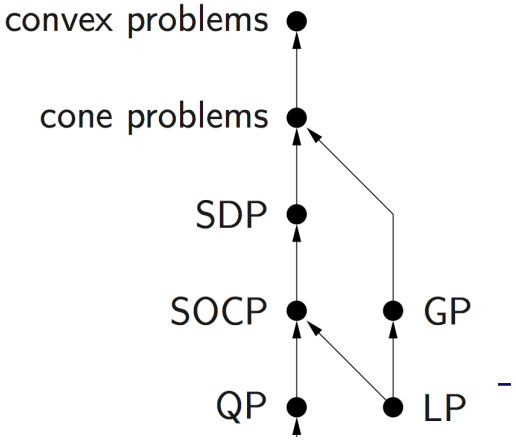
Summary of Optimization Problems

minimize	$f_0(x)$
subject to	$g_i(x) \le 0$, for i =1,...m
	$h_i(x) = 0$, for i =1,...,p.



The Optimization Family*





How to solve these problems?

minimize	$f_0(x)$
subject to	$g_i(x) \leq 0$, for i =1,...m
	$h_i(x) = 0$, for i =1,...,p.



Optimization Algorithms

Unconstrained Minimization

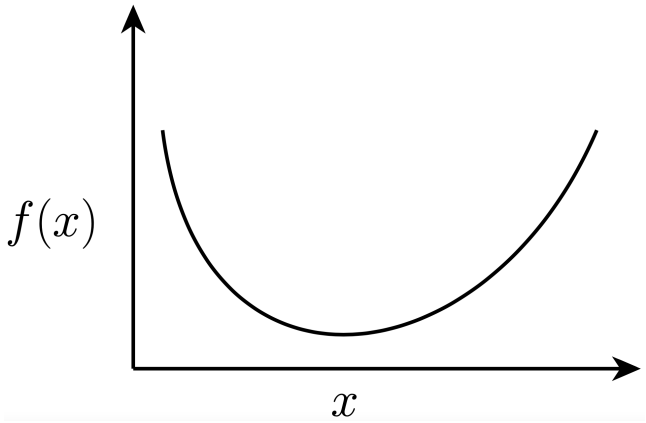
- Gradient descent, Newton's method

Constrained Minimization

- Interior-point Methods

Unconstrained Minimization

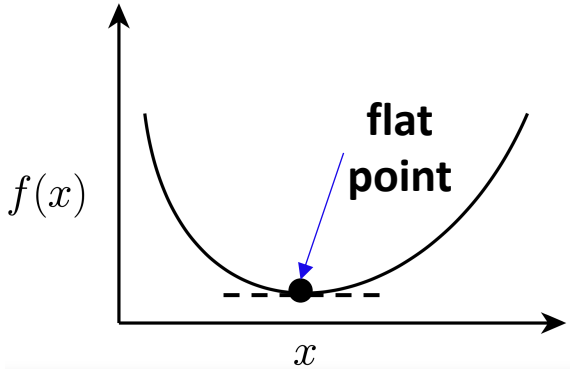
$$\text{minimize } f(x)$$



- f convex, twice continuously differentiable (hence **dom** f open)

How to Solve Unconstrained Minimization?

- Starting with the one dimensional case

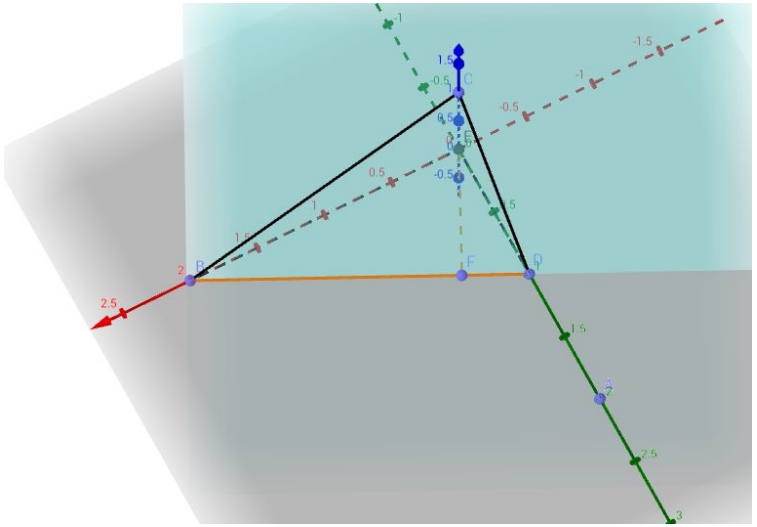


- To find minimum point $x^\star$, we can look at the derivative of the function $f'(x)$: any location where $f'(x) = 0$ will be a **"flat" point** in the function
- For convex problems, this is guaranteed to be a minimum.

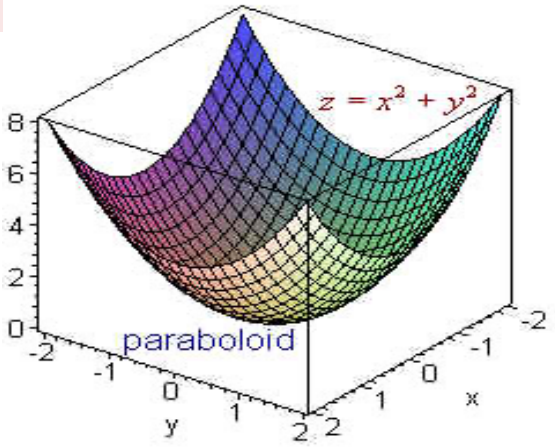
What is the "flat" point in the multi-dimensional case?

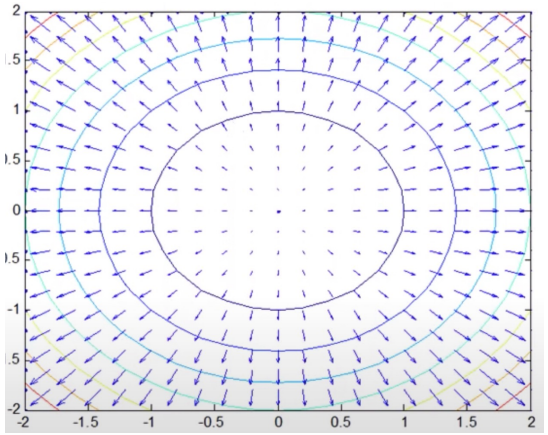
Gradient (梯度)

"direction and rate of the fastest increase"



https://www.zhihu.com/question/36301367





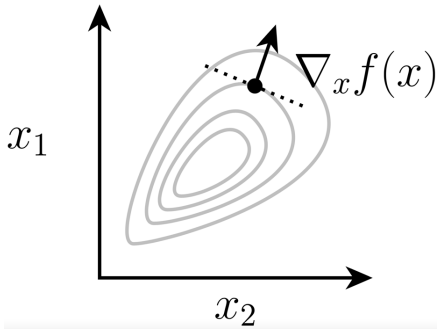
Contours for x^2+y^2 with gradients

Gradient

Definition

For f: $R^n \rightarrow R$, its gradient $\nabla_x f$: $R^n \rightarrow R^n$ at point $x = (x_1, \ldots, x_n)$ is defined as a vector containing partial derivatives with respect to each dimension.

$$\nabla_x f(x) = \begin{bmatrix} \frac{\partial f}{\partial x_1}(x) \\ \vdots \\ \frac{\partial f}{\partial x_n}(x) \end{bmatrix}$$



- For continuously differentiable f and unconstrained optimization, optimal point must have $\nabla_x f(x^*) = 0$

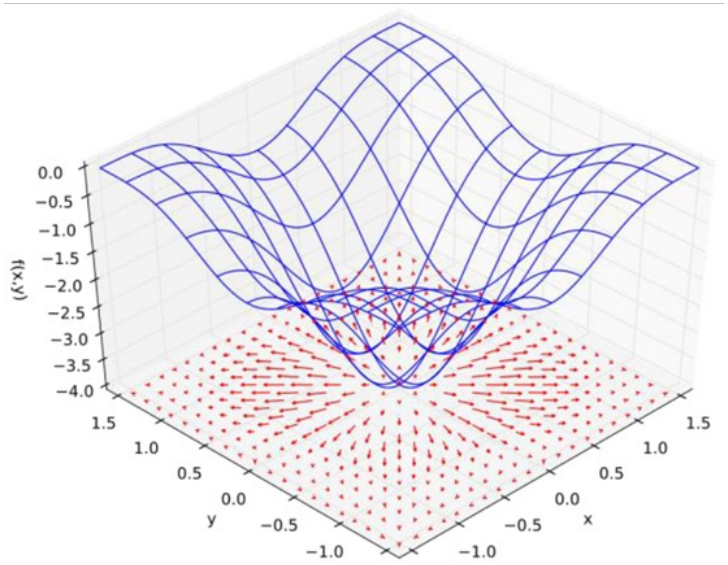
Gradient

Examples:

$$x = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix},$$

$$f(x) = 2x_1^2 x_2 - x_1 x_3^3$$

$$\nabla_x f(x) = \begin{bmatrix} \frac{\partial}{\partial x_1} f(x) \\ \frac{\partial}{\partial x_2} f(x) \\ \frac{\partial}{\partial x_3} f(x) \end{bmatrix} = \begin{bmatrix} 4x_1 x_2 - x_3^3 \\ 2x_1^2 \\ -3x_1 x_3^2 \end{bmatrix}$$

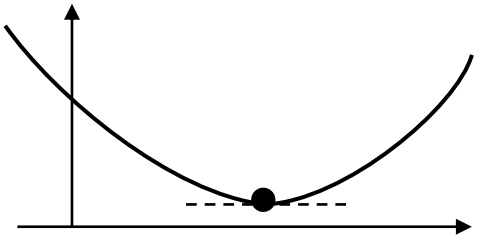


gradients for arbitrary function $f(x,y)$

How to Solve Unconstrained Minimization?

Direct Solution

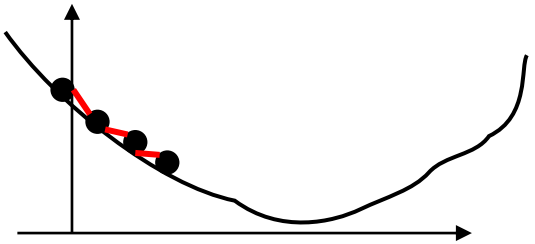
- In some cases, it is possible to analytically compute the $x^\star$ such that $\nabla_x f(x^\star) = 0$



Iterative methods

- More commonly the condition $\nabla_x f(x^\star) = 0$ will not have an analytical solution, require iterative methods
- Produce sequence of points $x^{(k)} \in \mathrm{dom}\, f$, k=0,1,… with

$$f(x^{(k)}) \rightarrow \mathrm{OPT(primal)}$$



Iterative Methods



Algorithms

- gradient descend
- steepest descend
- Newton's method
- …

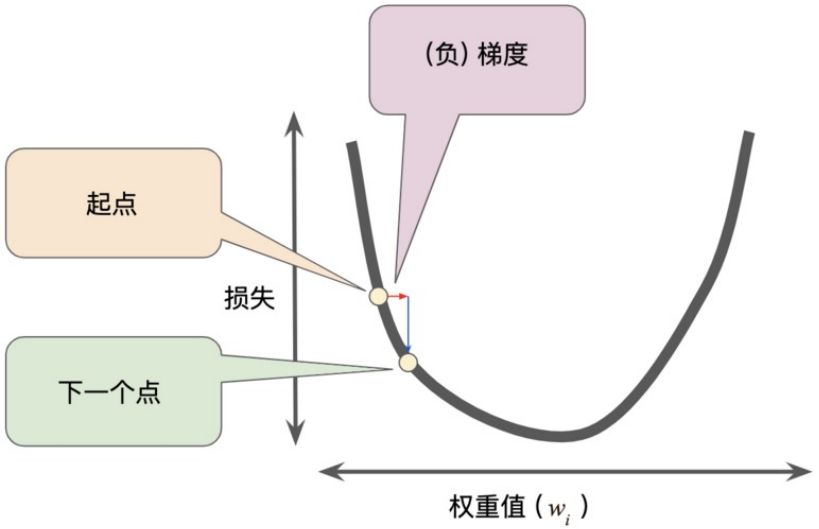
Gradient Descend

- Gradient not only indicates the optimality condition, but also points to the direction of "steepest ascent" for the function f.
- Motivates the gradient descent algorithm, which repeatedly takes steps in the direction of the negative gradient.

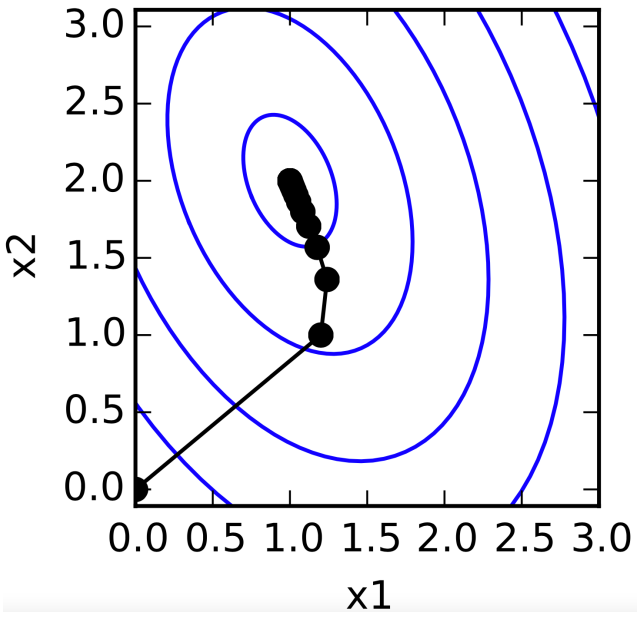
- **Goal**: $\min_x f(x)$
- **Iteration**:

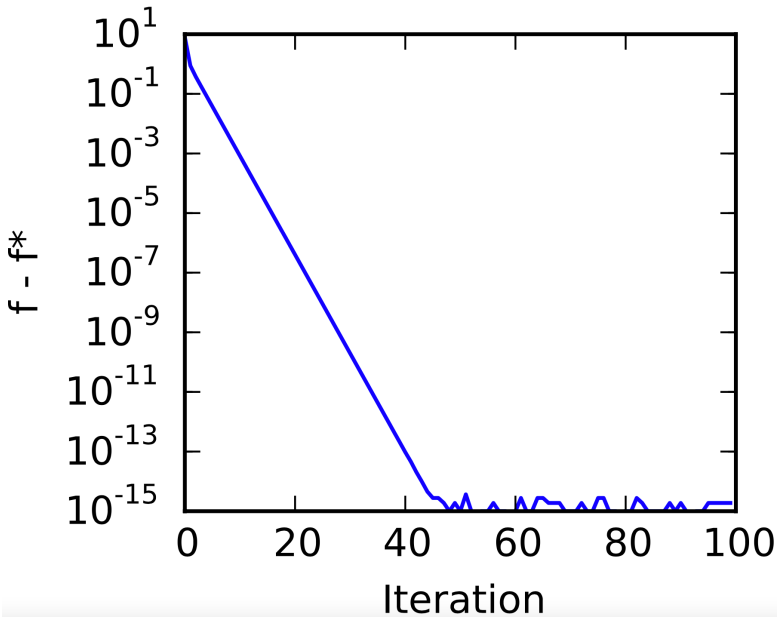
$$x_{t+1} = x_t - \eta_t \nabla f(x_t)$$

- η_t is the step size.



Gradient Descend





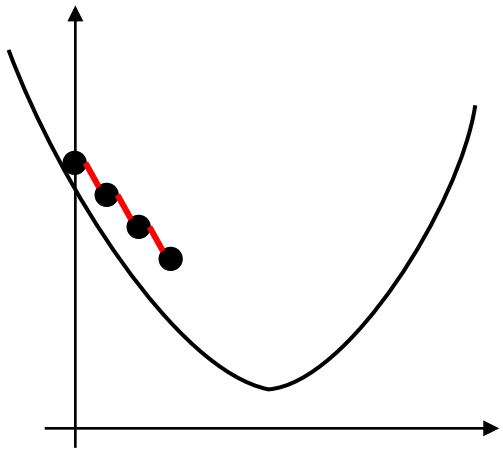
100 iterations of gradient descent on function

$$f(x) = 2x_1^2 + x_2^2 + x_1x_2 - 6x_1 - 5x_2$$

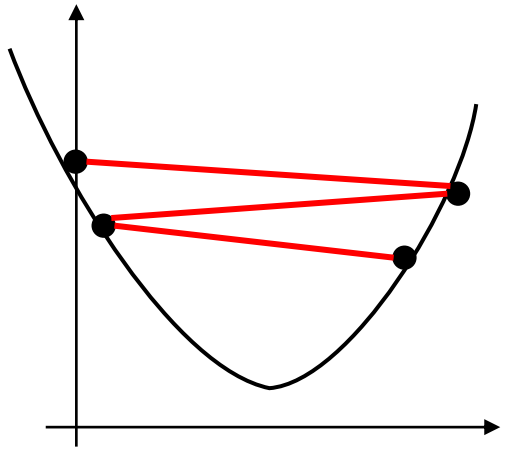
Choose Appropriate Step Size

$$x_{t+1} = x_t - \eta_t \nabla f(x_t)$$

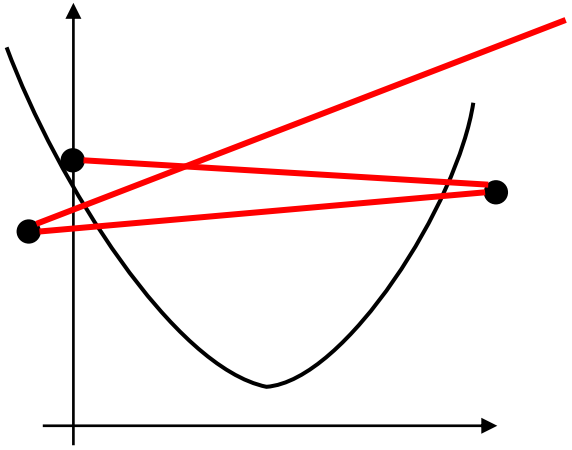
- If the step size is too big, the value of function can diverge.
- If the step size is too small, the convergence is too slow.



η too small:
slow progress

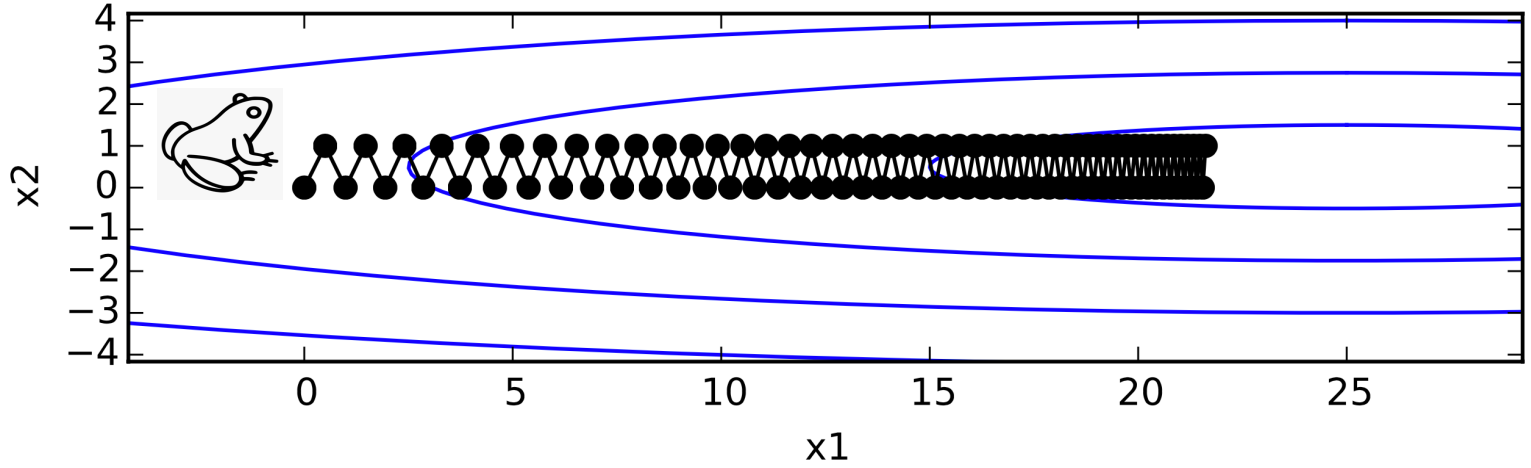


η too large:
oscillations



η much too large:
instability

Trouble with Gradient Descent



- Gradient descent on function

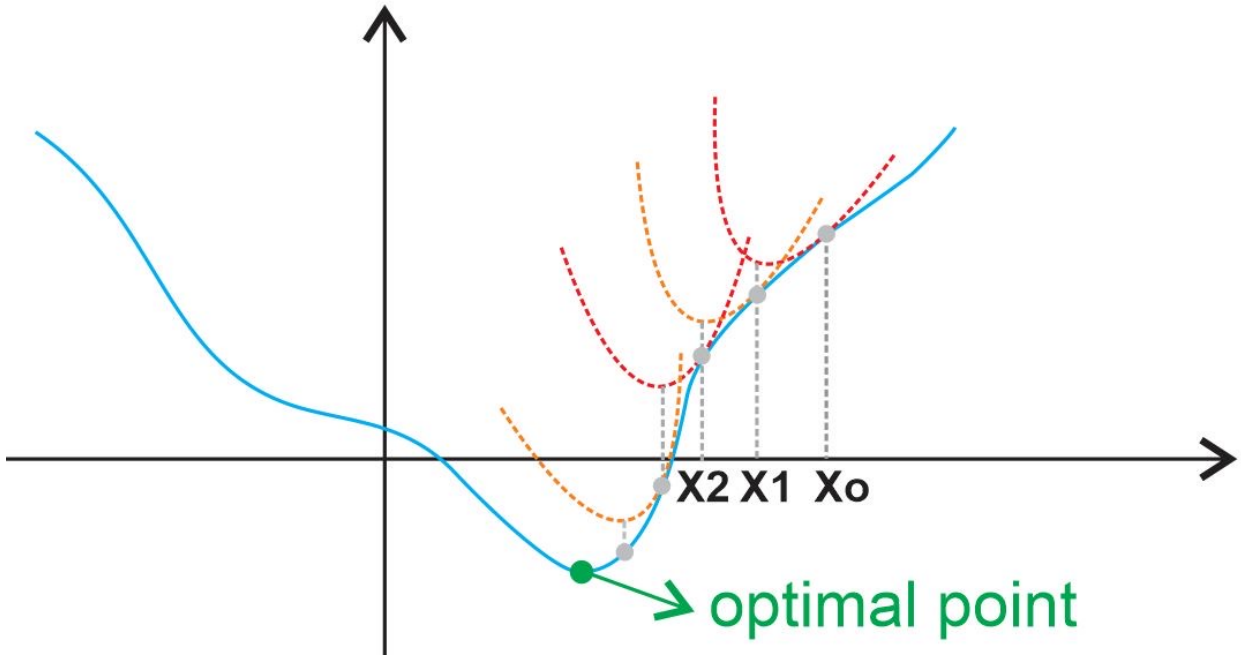
$$f(x) = 0.01x_1^2 + x_2^2 - 0.5x_1 - x_2$$

- Gradient is given by $(0.02x_1 - 0.5,\ 2x_2 - 1)$ which is very poorly scaled ($x_2 >> x_1$)
- Motivates approaches that "automatically" find the right scaling.

Newton's Method

- The next step moves to the minimal of the **second order approximation** of f

$$x_{t+1} = x_t - [\nabla^2 f(x_t)]^{-1} \nabla f(x_t)$$



Newton's Method

Interpretations

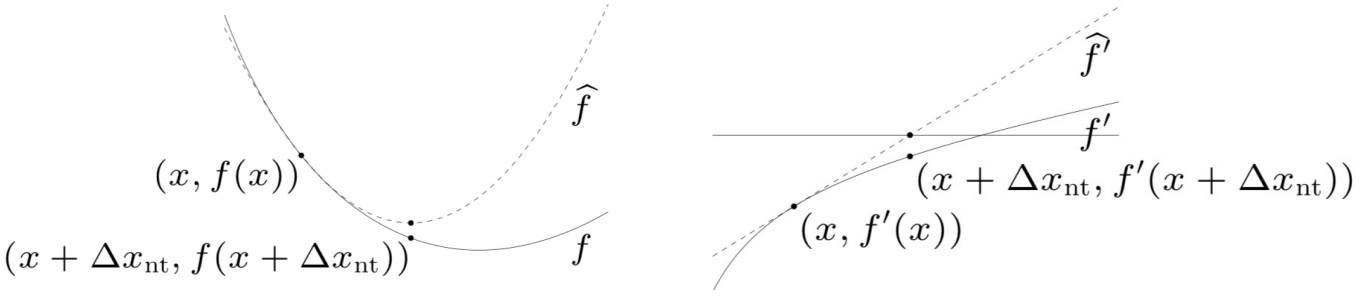
- $x+\Delta x_{\text{nt}}$ minimizes **second order approximation**

$$\hat{f}(x+v) = f(x) + \nabla f(x)^T v + \frac{1}{2} v^T \nabla^2 f(x) v$$

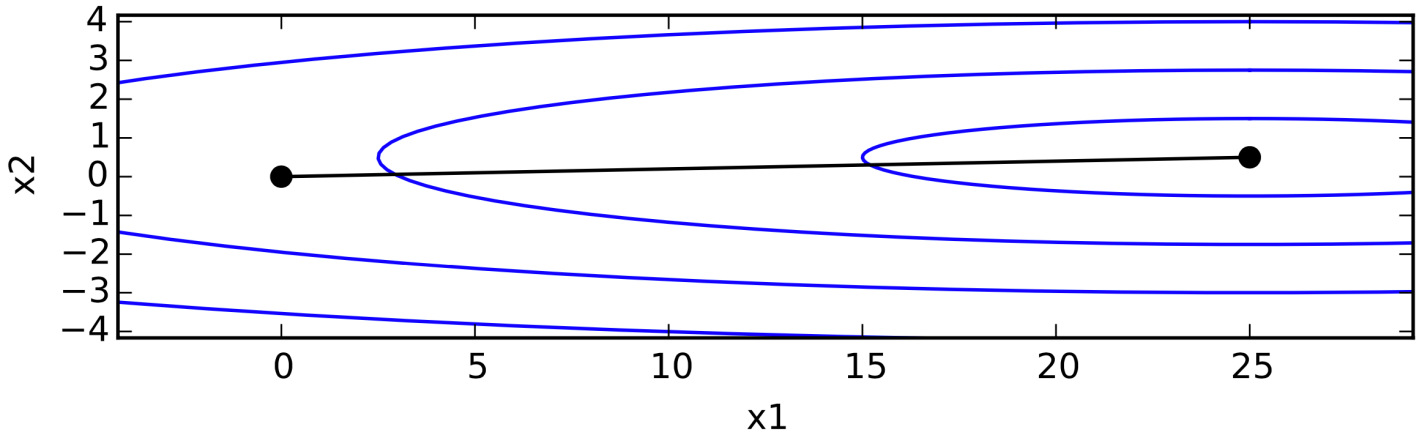
Let $\nabla \hat{f}(x+v) = 0$:

$$\nabla f(x+v) \approx \nabla \hat{f}(x+v) = \nabla f(x) + \nabla^2 f(x) v = 0$$

$$\Longrightarrow v^\star = \Delta x_{\text{nt}} = -\nabla^2 f(x)^{-1} \nabla f(x)$$



Newton's Method



$$f(x) = 0.01x_1^2 + x_2^2 - 0.5x_1 - x_2$$

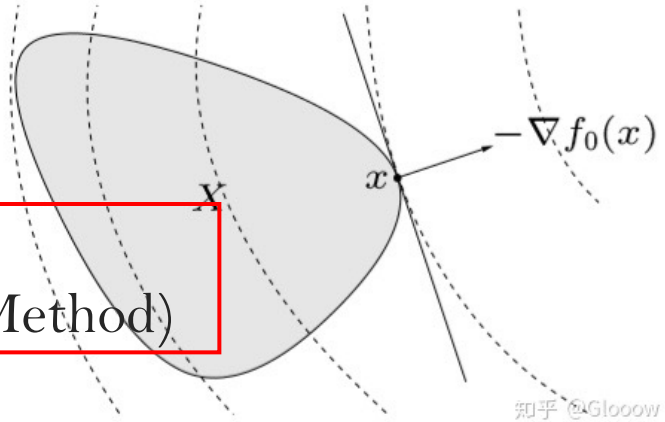
For our previous example, Newton's method finds the exact solution in a single step (holds true for any convex quadratic function)

- Newton's method is usually much faster than gradient descent

Constrained Minimization

Penalty Function Method (罚函数法)

- Quadratic Penalty Method
- Interior Point Method (内点法)
  (Logarithmic Penalty Method, Barrier Method)
- …



Augmented Lagrangian method

…

Inequality Constrained Minimization



- What about minimization with inequality constrains ?

$$\begin{aligned} &\text{minimize} && f_0(x) \\ &\text{subject to} && f_i(x) \le 0,\ i = 1,\ldots,m \\ & && Ax = b \end{aligned}$$

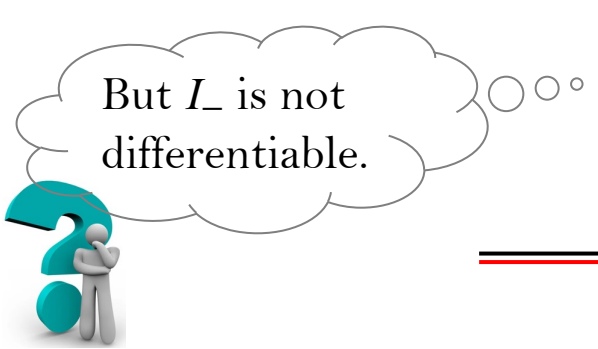
where f_i's are assumed to be convex & twice continuously differentiable

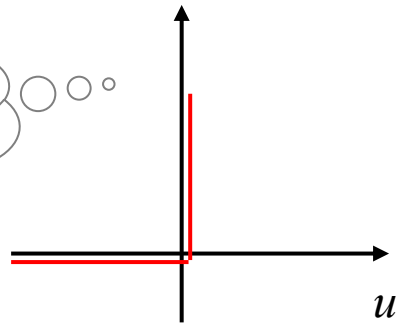
Trapping Points within the Feasible Set

- The inequality constraints can be made implicit by rewriting the objective with a penalty function $I_{-}()$.

$$\begin{aligned} &\min f_0(x) \\ &\text{s.t. } f_i(x) \le 0,\ i = 1,\ldots,m \\ &\quad Ax = b \end{aligned} \quad \Longrightarrow \quad \begin{aligned} &\min f_0(x) + \sum_{i=1}^{m} I_{-}(f_i(x)) \\ &\text{s.t. } Ax = b \end{aligned}$$

$$I_{-}(u) = \begin{cases} 0 & u \le 0 \\ \infty & \text{otherwise} \end{cases}$$





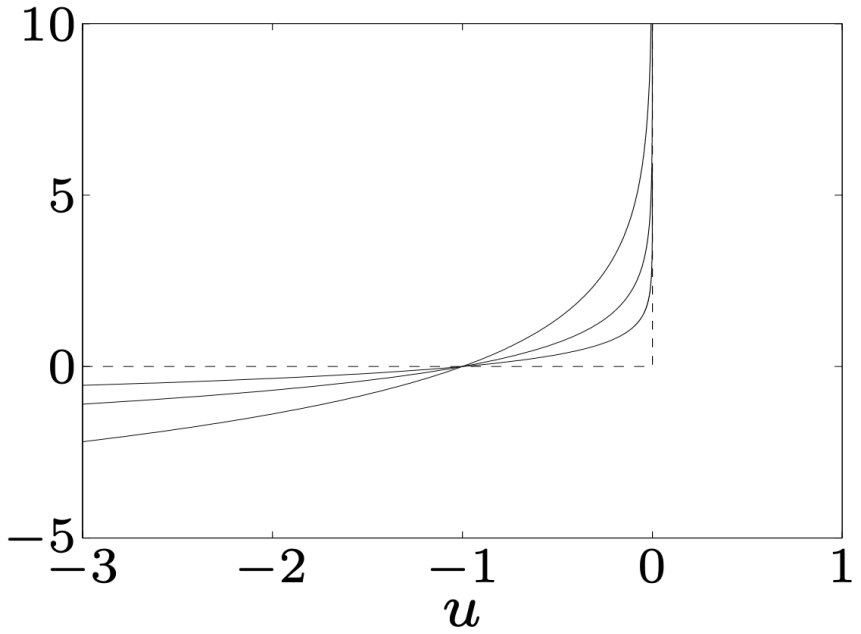
Idea: approximate I_{-} by some differentiable function.

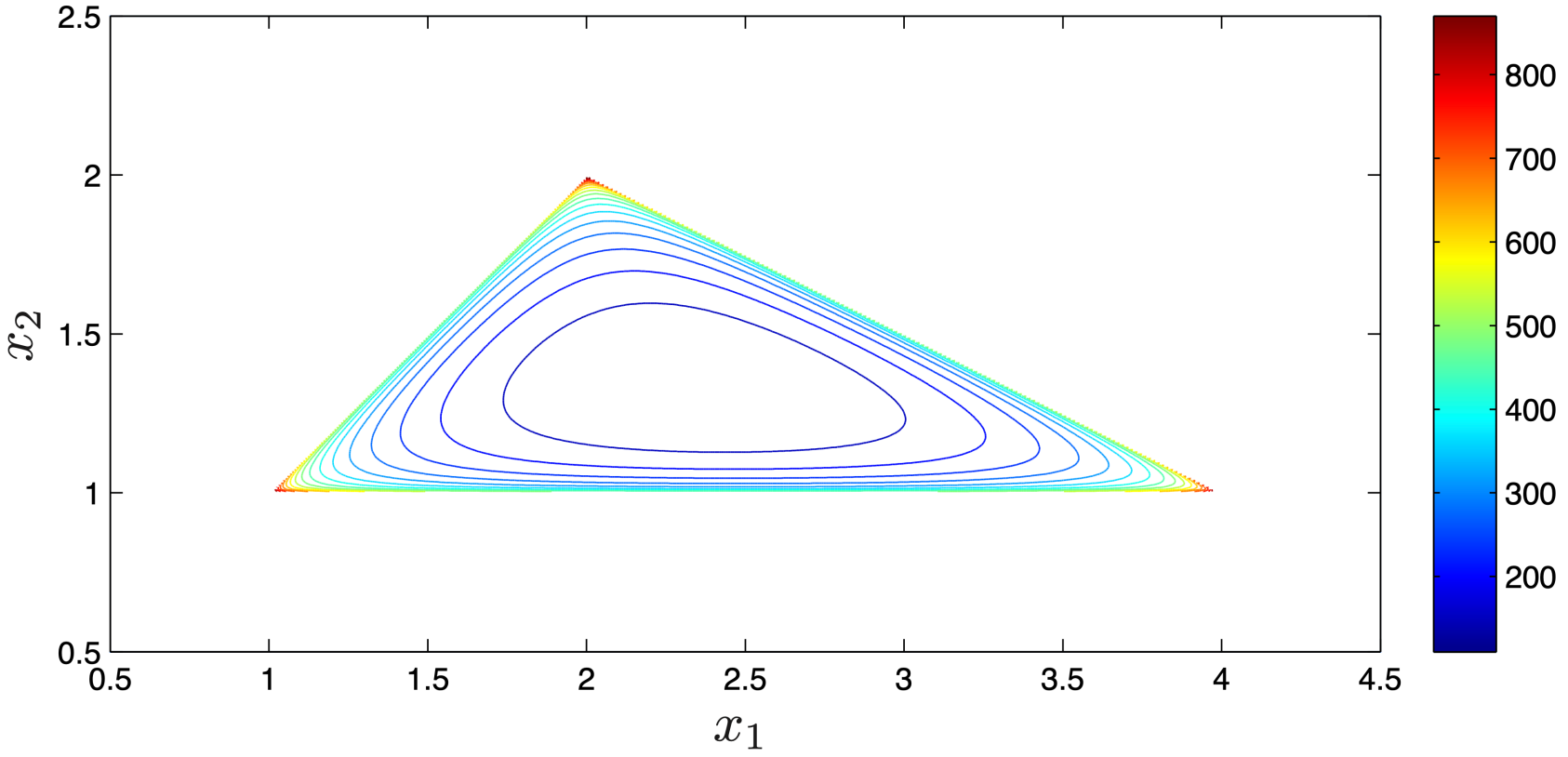
Logarithmic Barrier Function (对数闸函数)

- Approximate I_- by

$$I_-(u) = -\mu \log(-u), \quad \operatorname{dom} I_- = \{x \in \mathbb{R} \mid x < 0\}$$

where $\mu > 0$ is a parameter that controls the accuracy of the approx.





Example: A set of linear inequalities $b_i - a_i^T x \leq 0$, $i = 1,...,m$.

The corresponding approx. is

$$-\mu \sum_{i=1}^{m} \log(a_i^T x - b_i)$$

Logarithmic Barrier Function

- The original problem can be approximated as:

$$\begin{aligned} \min \ & f_0(x) + \mu\phi(x) \\ \text{s.t.} \ & Ax = b \end{aligned}$$

with $\phi(x) = -\sum_{i=1}^{m} \log(-f_i(x)), \quad \mathbf{dom}\, \phi = \{x \mid f_1(x) < 0, \ldots, f_m(x) < 0\}$

Central Path

- The log barrier approximation

$$\begin{boxed} \min \ f_0(x) + \mu\phi(x) \\ \text{s.t. } Ax = b \end{boxed} \qquad (\star)$$

  - is an accurate approximation of the original problem for $\mu \to 0$ (with $\mu > 0$)
  - becomes difficult to minimize as $\mu \to 0$ (at least for Newton's method)



Idea: start with a large μ, iteratively reduce μ until a desired solution accuracy is reached.

Central Path

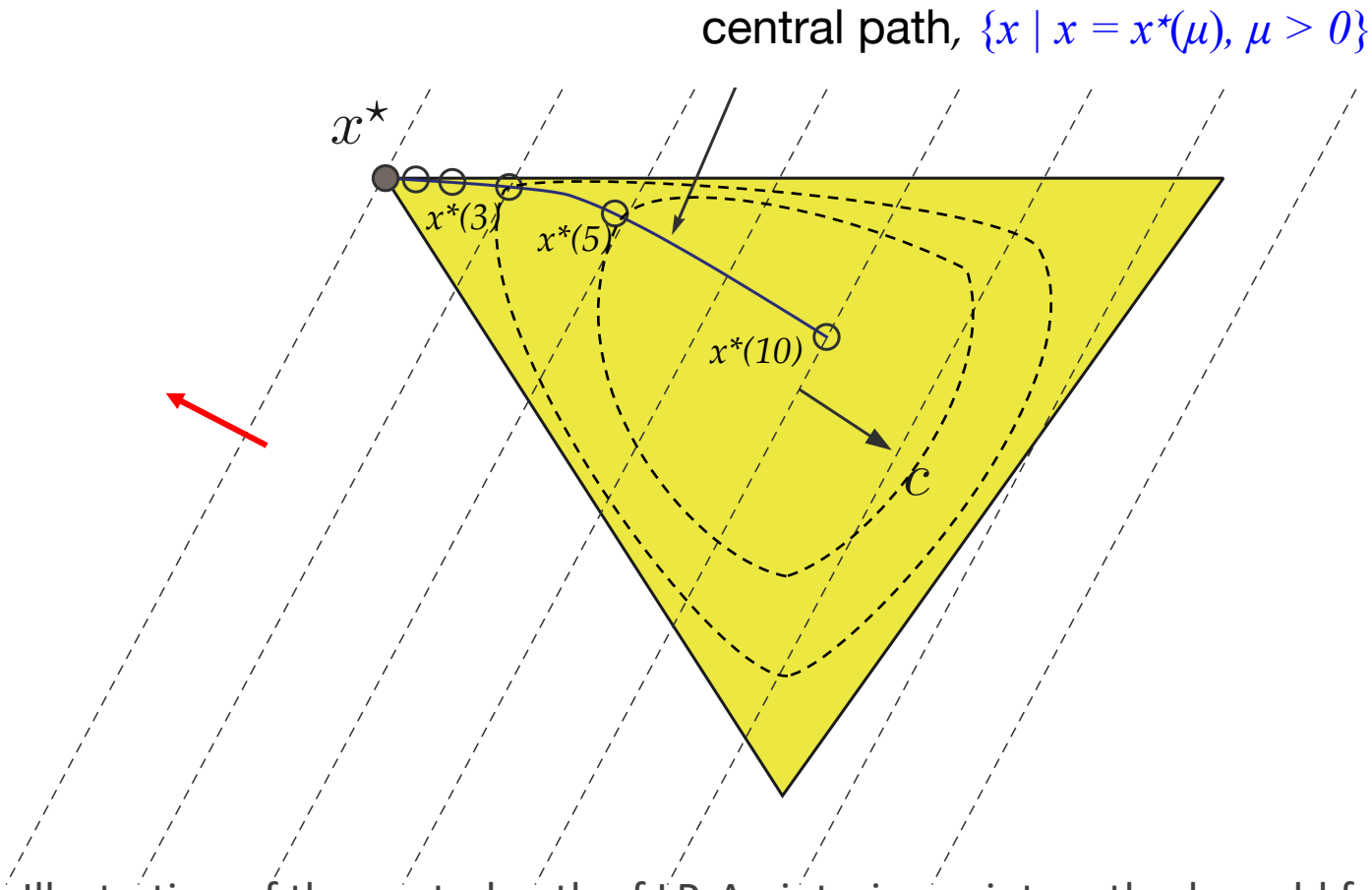


Illustration of the central path of LP. An interior-point method would follow the central path to iteratively approach the optimal solution.

Barrier Method (Path Following Method)

Path Following Method

given an initial strictly feasible point x, μ, $\epsilon > 0$, & $\beta < 1$.

repeat

1. *Centering step.* Starting at x, use Newton's method to solve
$$x^\star(\mu) = \min_x \mu f_0(x) + \phi(x) \text{ s.t. } Ax = b,$$
2. *Update.* $x := x^\star(\mu)$.
3. *Stopping criterion.* **quit** if $m\mu < \epsilon$.
4. *Target shifting.* $\mu := \beta\mu$.

The barrier method is also known as the **interior point method** because it requires that the variables are always within the feasible set (i.e., do not violate inequality constraints).

案例：云计算资源分配

一家云计算服务提供商需要分配计算资源来运行两种类型的任务：**数据处理任务**和**机器学习任务**。每个**数据处理任务**的利润为$200，需要**1单位CPU** 和 **3单位内存**。每个**机器学习任务**的利润为$100，需要 **2单位CPU**和**1单位内存**。该云服务提供商当前可用的资源总量为：**6,000单位CPU**和**9,000单位内存**

问：为了最大化利润，应该分配多少资源给**数据处理任务**和**机器学习任务**？

案例：云计算资源分配

设：

x = 数据处理任务的数量

y = 机器学习任务的数量

目标函数：

maximize $z=200x+100y$ (最大化利润)

约束条件：

$x+2y \leq 6000$ (CPU 约束)

$3x+y \leq 9000$ (内存 约束)

$x \geq 0, y \geq 0$ (非负约束)

案例：云计算资源分配

使用Python的 scipy.optimize.linprog 来求解这个线性规划问题。

```python
from scipy.optimize import linprog

# 目标函数系数（注意：linprog默认是最小化问题，所以取负号）
c = [-200, -100]  # 利润函数：200x1 + 100x2

# 不等式约束矩阵
A = [
    [1, 2],  # CPU 约束：x1 + 2x2 <= 6000
    [3, 1]   # 内存约束：3x1 + x2 <= 9000
]

# 不等式约束右侧
b = [6000, 9000]

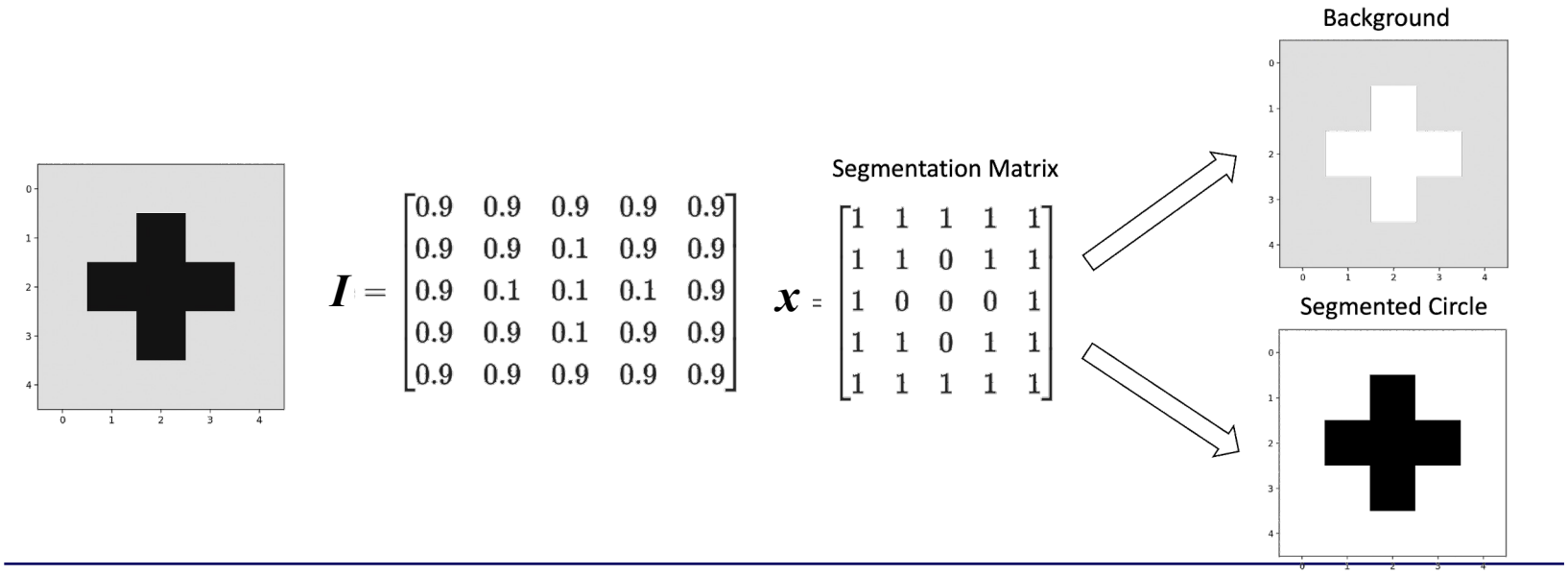
# 变量边界（非负约束）
x_bounds = (0, None)  # x1 >= 0, x2 >= 0
```

```python
# 求解线性规划问题
result = linprog(c, A_ub=A, b_ub=b, bounds=[x_bounds, x_bounds], method='highs')

# 输出结果
if result.success:
    x1_opt = result.x[0]
    x2_opt = result.x[1]
    max_profit = -result.fun  # 因为目标函数取了负号
    print(f"最优解: 数据处理任务 = {x1_opt:.2f}, 机器学习任务 = {x2_opt:.2f}")
    print(f"最大利润: ${max_profit:.2f}")
else:
    print("未找到最优解")
```

案例：图像分割

在计算机中，我们经常需要将图片的目标和背景分离。给定一张灰度图片I，其中每个像素$I_{ij} \in [0,1]$表示该点的灰度值（0：黑，1：白）。为了将目标分割，需要一个同大小的分割矩阵$x_{ij} \in \{0,1\}$，$x_{ij}=1$表示该像素属于目标，$x_{ij}=0$表示该像素属于背景。目标是找到一个分割矩阵x使得分割后的目标和背景与真实目标背景的差距最小。



$$\boldsymbol{I} = \begin{bmatrix} 0.9 & 0.9 & 0.9 & 0.9 & 0.9 \\ 0.9 & 0.9 & 0.1 & 0.9 & 0.9 \\ 0.9 & 0.1 & 0.1 & 0.1 & 0.9 \\ 0.9 & 0.9 & 0.1 & 0.9 & 0.9 \\ 0.9 & 0.9 & 0.9 & 0.9 & 0.9 \end{bmatrix} \quad \boldsymbol{x} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

案例：图像分割

问题形式化

min $\sum_{i,j} x_{ij} \times (1 - 2I[i,j])$
s.t. $x_{ij} \in \{0,1\}$ for all pixels (i, j)

relax ⇨

min $\sum_{i,j} x_{ij} \times (1 - 2I[i,j])$
s.t. $x_{ij} > 0$, $x_{ij} < 1$, for all pixels (i, j)

- 当像素值I较大（接近1）时，$(1-2I)$为负，优化器会倾向于使x接近1（目标区域）。
- 当I较小（接近0）时，$(1-2I)$为正，优化器会倾向于使x接近0（背景区域）。

案例：图像分割

使用Python的cvxpy求解该优化问题

```
import numpy as np
import cvxpy as cp
import matplotlib.pyplot as pl

# 生成示例图像
def generate_example_image
    image = np.zeros(size)
    # 添加目标区域
    image[20:60, 30:70] = 0.8  #
    # 添加噪声
    image += np.random.norma
    image = np.clip(image, 0, 1)
    return image
```

```
# 图像分割优化
def image_segmentation(image):
    # 将图像展平为一维向量
    I = image.flatten()
    n = len(I)
    # 定义优化变量
    x = cp.Variable(n)
    # 定义目标函数
    objective = cp.Minimize(cp.sum(x @ (1
I)))
    # 定义约束条件
    constraints = [0 <= x, x <= 1]
    # 求解优化问题
    problem = cp.Problem(objective, const
    problem.solve()
    # 将结果重新调整为图像形状
    x_opt = x.value.reshape(image.shape)
    return x_opt
```

```
if __name__ == "__main__":
    # 生成示例图像
    image = generate_example_image()
    # 进行图像分割
    x_opt = image_segmentation(image)
    # 显示结果
    plt.figure(figsize=(10, 5))
    # 显示原始图像
    plt.subplot(1, 2, 1)
    plt.imshow(image, cmap='gray')
    plt.title('Original Image')
    plt.axis('off')

    # 显示分割结果
    plt.subplot(1, 2, 2)
    plt.imshow(x_opt, cmap='gray')
    plt.title('Segmentation Result')
    plt.axis('off')

    plt.show()
```

案例：投资组合优化

某银行现有A、B两类理财产品售价分别为1万元/份和2万元/份，到期后的**预期收益**分别是2万元/份和1万元/份。某客户有现金10万元，拟分别投资A、B产品x_1，x_2份（x_1，x_2可以是连续值）。考虑到投资风险，该客户作了行业调研，评估出该投资组合(Portfolio)的**预估风险**为$\frac{1}{2}(x_1^2 + x_2^2)$万元。

问题：总现金投入下，如何分配两个产品份额，以最大化综合收益（综合收益=预期收益-预估风险）

案例：投资组合优化

(1) 优化问题表示

客户的目标是在总现金投入下最大化综合收益，综合收益定义为预期收益减去预估风险。具体表示如下：

- **决策变量：**
  - x_1：投资A类理财产品的份数
  - x_2：投资B类理财产品的份数
- **目标函数：**

综合收益=预期收益−预估风险=$(2x_1+x_2)-1/2(x_1^2+x_2^2)$ 即：

$$\max x_1,x_2 \ \ 2x_1+x_2-1/2(x_1^2+x_2^2)$$

- **约束条件：**

$$x_1+2x_2\leq 10$$

其中，$x_1\geq 0$ 和 $x_2\geq 0$。

案例：投资组合优化

(2) 梯度下降法优化目标函数

1. 初始化： 选择初始值 $x_1^{(0)}$ 和 $x_2^{(0)}$，设定步长 η。

2. 计算梯度： $\nabla f(x_1,x_2)=(\partial f/\partial x_1,\partial f/\partial x_2)$

其中，$\partial f/\partial x_1=2-x_1$　$\partial f/\partial x_2=1-x_2$

3. 更新步骤：

$$x_1^{(k+1)}=x_1^{(k)}+\eta\cdot\partial f/\partial x_1=x_1^{(k)}+\eta(2-x_1^{(k)})$$
$$x_2^{(k+1)}=x_2^{(k)}+\eta\cdot\partial f/\partial x_2=x_2^{(k)}+\eta(1-x_2^{(k)})$$

4. 投影到可行域：

在每次更新后，需要确保 x_1 和 x_2 满足约束条件 $x_1+2x_2\leq 10$ 以及 $x_1\geq 0$ 和 $x_2\geq 0$。如果更新后的值不满足约束条件，则需要将其投影到可行域内

5. 迭代： 重复更新步骤直到收敛。

案例：投资组合优化

采用python求解该优化问题

```python
import numpy as np

# 定义目标函数
def objective_function(x1, x2):
    return 2 * x1 + x2 - 0.5 * (x1**2 + x2*

# 定义梯度
def gradient(x1, x2):
    df_dx1 = 2 - x1  # 对x1的偏导数
    df_dx2 = 1 - x2  # 对x2的偏导数
    return np.array([df_dx1, df_dx2])

# 投影函数，确保满足约束条件
def project_to_feasible_region(x1, x2):
    # 确保x1和x2非负
    x1 = max(x1, 0)
    x2 = max(x2, 0)
    # 确保满足约束条件 x1 + 2*x2 <= 10
    if x1 + 2 * x2 > 10:
        # 如果超出约束，投影到约束边界
        x1 = max(x1, 0)
        x2 = max((10 - x1) / 2, 0)
    return x1, x2
```

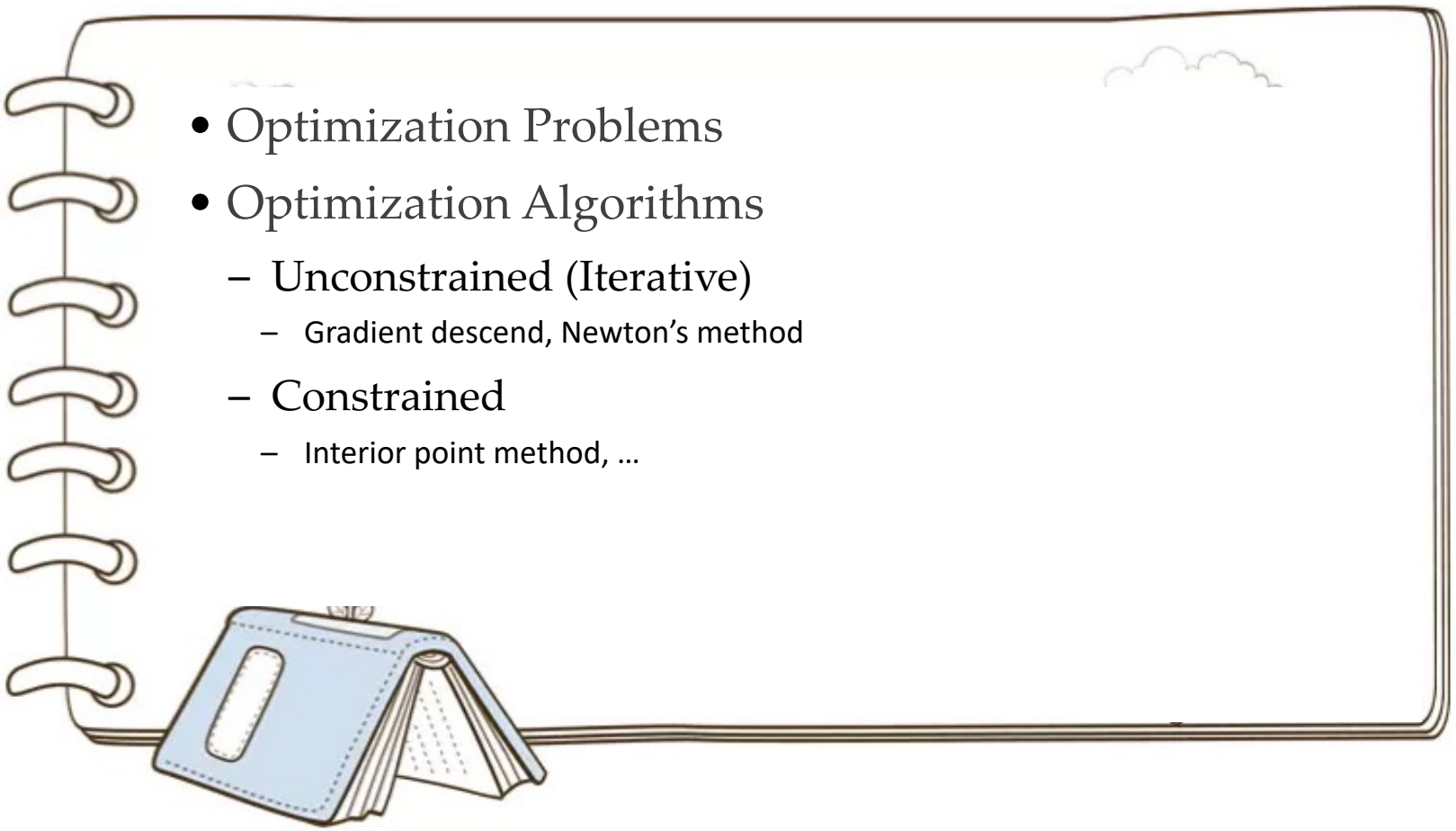
```python
# 梯度下降法
def gradient_descent(initial_x1, initial_x2, learning_r
ate, max_iterations, tolerance):
    x1, x2 = initial_x1, initial_x2
    for i in range(max_iterations):
        # 计算梯度
        grad = gradient(x1, x2)
        # 更新变量
        x1_new = x1 + learning_rate * grad[0]
        x2_new = x2 + learning_rate * grad[1]
        # 投影到可行域
        x1_new, x2_new = project_to_feasible
1_new, x2_new)
        # 检查收敛条件
        if np.linalg.norm([x1_new - x1, x2_new
olerance:
            break
        # 更新变量
        x1, x2 = x1_new, x2_new
    return x1, x2, objective_function(x1, x2)
```

```python
# 参数设置
initial_x1 = 0  # 初始值
initial_x2 = 0  # 初始值
learning_rate = 0.1  # 步长
max_iterations = 1000  # 最大迭代次数
tolerance = 1e-6  # 收敛容忍度

# 运行梯度下降法
x1_opt, x2_opt, max_value = gradient_des
cent(initial_x1, initial_x2, learning_rate, m
ax_iterations, tolerance)

# 输出结果
print(f"最优解: x1 = {x1_opt:.4f}, x2 = {x2_
opt:.4f}")
print(f"最大综合收益: {max_value:.4f}")
```

Summary



- Optimization Problems
- Optimization Algorithms
  - Unconstrained (Iterative)
    - Gradient descend, Newton's method
  - Constrained
    - Interior point method, …